

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

ОТЧЁТ

по лабораторной работе № 2

«Мышь в лабиринте» — обучение с подкреплением

по дисциплине «Объектно-ориентированное программирование на языке
Java»

Выполнил

студент гр. 5130902/40202

А. А. Минязов

Проверил

старший преподаватель

Д. С. Хасанов

Санкт-Петербург

2026

СОДЕРЖАНИЕ

1 Постановка задачи	3
1.1 Цель работы	3
1.2 Суть обучения с подкреплением	3
2 Описание модели	5
2.1 Лабиринт	5
2.2 Награды	5
2.3 Алгоритм Q-learning	6
2.4 Попытки и остановка обучения	7
2.5 Подбор параметров	7
2.6 Ограничение модели	8
3 Проектирование	10
3.1 Диаграмма прецедентов	11
3.2 Диаграммы деятельности	12
3.3 Диаграммы классов	14
3.4 Диаграмма взаимодействия	17
3.5 Диаграммы состояний	18
3.6 Диаграмма компонентов	20
4 Реализация	21
4.1 Средства разработки	21
4.2 Структура программы	21
4.3 Применение принципов ООП	21
4.4 Оконное приложение	22
4.5 Защита от некорректного ввода	23
5 Демонстрация визуализации	24
Заключение	32
Приложение А. Текст программы	33

1 Постановка задачи

Мышка ищет в конце лабиринта самый большой наградной сыр (+Z). Мышь должна пить воду; вода имеет меньшую награду (+x). Мышь хочет избежать электротравмы (−y). Без обратной связи мышь не знает, какое действие лучше сделать на следующем шаге.

Обратная связь — это подкрепление. На каждом шаге мышь выбирает направление движения, а окружающая среда сообщает величину выигрыша или проигрыша. Мышь должна знать сумму выигрыша или проигрыша.

Программа должна генерировать лабиринт любого размера; размер и схему лабиринта задаёт пользователь. Результат работы программы необходимо показать визуально.

1.1 Цель работы

Цель работы — разработать на языке Java программу, в которой агент (мышь) без знания карты учится проходить лабиринт по наградам среды, и освоить на её примере принципы объектно-ориентированного программирования и основы обучения с подкреплением.

1.2 Суть обучения с подкреплением

В обучении с подкреплением агент не получает готовых правил поведения. Он действует в среде, после каждого действия получает от неё число — награду — и сам выясняет, какие действия в каких ситуациях выгоднее. Термины, используемые в работе:

- **состояние** — где находится мышь (клетка лабиринта);
- **действие** — шаг в одном из четырёх направлений: вверх, вправо, вниз, влево;
- **награда** — число, которое среда сообщает после шага (сыр, вода, ток, плата за шаг, удар о стену);
- **попытка (эпизод)** — путь мыши от старта до сыра или до лимита шагов; выученное между попытками сохраняется;

- **стратегия** — правило выбора действия; после обучения — «в каждой клетке идти туда, где ожидается наибольшая сумма наград».

Для решения задачи выбран алгоритм Q-learning — один из основных алгоритмов обучения с подкреплением, не требующий знания карты среды.

2 Описание модели

2.1 Лабиринт

Лабиринт — прямоугольник из $W \times H$ клеток (от 1×2 до 100×100). Стены тонкие и располагаются между клетками; граница лабиринта — всегда стена. В клетке может находиться вода или ток; отдельно задаются клетка старта и клетка с сыром. Стены хранятся в двух логических массивах: «стена справа от клетки» и «стена снизу от клетки», поэтому одна стена не может иметь двух противоречивых записей.

Случайный лабиринт строится в три этапа:

- ставятся все стены, затем обход в глубину со случайным выбором соседа пробивает проходы — получается «идеальный» лабиринт, в котором к любой клетке ведёт ровно один путь;
- убирается заданный процент оставшихся стен — появляются обходные пути, и мыши есть из чего выбирать (например, между коротким путём через ток и обходом);
- вода и ток ставятся в случайные клетки, кроме старта и сыра.

Все случайные решения принимает один генератор, созданный из зерна (seed), поэтому одинаковый seed всегда даёт одинаковый лабиринт. Схему лабиринта пользователь может изменить в редакторе: поставить или убрать стену, воду, ток, перенести старт и сыр.

2.2 Награды

После каждого шага среда (класс `Attempt`) сообщает мыши награду (таблица 1). Сумму наград за попытку мышь знает всегда; она выводится на экран и на график.

Таблица 1 — Награды по умолчанию

Событие	Награда	Пояснение
Сыр	+100 (+Z)	попытка заканчивается
Вода	+10 (+x)	выпивается и до конца попытки не даёт награды; в следующей попытке снова на месте

Событие	Награда	Пояснение
Удар током	−50 (−y)	каждый раз, когда мышь входит в клетку
Каждый шаг	−1	короткий путь выгоднее длинного
Удар о стену	−5	мышь остаётся на месте

Вода выпивается, иначе мышь научилась бы бесконечно «доить» воду, бегая туда и обратно, и не шла бы к сыру. Награды можно изменить в окне параметров; программа проверяет, что сыр — самая большая награда, вода меньше сыра, а ток не является наградой.

2.3 Алгоритм Q-learning

Мышь хранит Q-таблицу — для каждой клетки и каждого из четырёх направлений число $Q[s][a]$: оценку того, сколько очков в сумме она получит, если из клетки s шагнёт в направлении a и дальше будет действовать наилучшим образом. Вначале все числа равны нулю.

Выбор шага (ϵ -жадная стратегия). С вероятностью ϵ мышь шагает в случайном направлении, чтобы попробовать новые пути; иначе — в направлении с наибольшим числом Q (из нескольких равных — случайно). Без случайных шагов мышь навсегда осталась бы на первом найденном пути, даже если обход лучше.

Обучение. После шага мышь исправляет одно число — у стрелки, по которой прошла:

$$Q[s][a] \leftarrow Q[s][a] + \alpha \cdot (r + \gamma \cdot \max_{a'} Q[s'] - Q[s][a]),$$

где r — полученная награда, s' — клетка, в которой мышь оказалась, $\max Q[s']$ — наибольшее число в ней (у сыра будущего нет, и это слагаемое равно нулю), α — скорость обучения, γ — коэффициент дисконтирования. Смысл формулы: шаг хорош, если за него сразу дали награду или он привёл туда, откуда уже известен хороший путь. С каждой попыткой ценность сыра уходит на шаг дальше от него, пока не дойдёт до старта.

Коэффициент γ определяет, какая доля ценности сохраняется при удалении от награды на один шаг. Он должен быть близок к единице: при $\gamma =$

0.9 сыр за 70 шагов стоит для мыши на старте $100 \cdot 0.9^{70} \approx 0.06$ очка — меньше платы за один шаг, и мышь не «чувствует» сыр вовсе. При $\gamma = 0.995$ сыр за 70 шагов стоит около 70 очков.

Доля случайных шагов линейно уменьшается от $\varepsilon = 0.3$ до 0.01 за первые 300 попыток: вначале мышь ничего не знает и должна исследовать лабиринт, затем в основном пользуется выученным.

2.4 Попытки и остановка обучения

Попытка заканчивается, когда мышь дошла до сыра или сделала максимальное число шагов (по умолчанию 4000). Лимит нужен, потому что в первых попытках мышь может бродить очень долго, а если сыр отгорожен стенами — вечно. После окончания попытки её сумма записывается в историю, мышь возвращается на старт, выученная таблица сохраняется.

Само обучение останавливает пользователь: кнопкой «Пауза», заданием числа попыток («Обучить N») или автостопом. **Лучший путь** — путь мыши без случайных шагов, каждый раз по стрелке с наибольшим числом. Автостоп срабатывает, когда лучший путь доходит до сыра и не меняется заданное число попыток подряд (по умолчанию 50); программа сообщает, за сколько попыток мышь выучила маршрут.

2.5 Подбор параметров

Награды заданы условием задачи и не подбирались. Параметры обучения подобраны перебором. В первом раунде проверено 720 наборов ($\alpha \in \{0.1; 0.3; 0.5; 0.8; 1\}$, $\gamma \in \{0.9; 0.95; 0.98; 0.99\}$, начальное $\varepsilon \in \{0.1; 0.2; 0.3; 0.5\}$, три скорости убывания ε , три лимита шагов) на 30 случайных лабиринтах размером 10×10 , 15×15 и 20×20 с большим количеством воды и тока, по 1000 попыток. Качество набора оценивалось тем, в скольких лабиринтах лучший путь в конце доходит до сыра, и тем, сколько очков он не добывает до лучшего найденного результата. Главным фактором оказался γ (таблица 2).

Таблица 2 — Влияние γ (среднее по всем остальным параметрам, 30 лабиринтов)

γ	Мышь нашла путь, лабиринтов из 30
0.9	13.9
0.95	15.0
0.98	21.8
0.99	24.7

Во втором раунде проверено 216 наборов вокруг лучших, включая $\gamma = 0.995$ и 0.999 . Итоговые значения проверены на 60 других лабиринтах размером до 30×30 , не участвовавших в подборе (таблица 3).

Таблица 3 — Параметры обучения по умолчанию

Параметр	Пробные значения	Подобранные
α — скорость обучения	0.5	0.2
γ — дисконтирование	0.95	0.995
ϵ — доля случайных шагов	$0.3 \rightarrow 0.01$ за 300 попыток	$0.3 \rightarrow 0.01$ за 300 попыток
Лимит шагов в попытке	1000	4000
Мышь нашла путь (60 проверочных лабиринтов)	25	60
Недобор до лучшего результата, очков	118.4	2.4

Лабиринт при запуске программы (12×9 , 4 клетки с водой, 6 с током, 20 % обходов, seed 9) выбран так, что кратчайший путь к сыру проходит через ток, а обученная мышь обходит его и заходит за водой.

2.6 Ограничение модели

Состояние мыши — только клетка: мышь не знает, какая вода в лабиринте уже выпита. Пока шаги стоят заметно (плата -0.5 и дороже) или γ заметно меньше единицы, это не мешает. При почти бесплатных шагах и γ , близком к единице, ошибки оценок ничем не гасятся: числа в таблице раздуваются, и мышь может не выучить маршрут. Хранить в состоянии всю выпитую воду нельзя: таблица выросла бы в 2^n раз, где n — число клеток с водой. Поэтому комбинация $\gamma = 1$ и бесплатных шагов запрещена (мыши в

этом случае всё равно, как долго идти), а при почти бесплатных шагах окно параметров предупреждает пользователя и применяет настройки только после повторного подтверждения.

3 Проектирование

Система описана UML-диаграммами шести видов: прецедентов, деятельности, классов, взаимодействия (последовательности), состояний и компонентов. Диаграммы построены в PlantUML.

3.1 Диаграмма прецедентов

Пользователь строит и редактирует лабиринт, задаёт параметры, управляет обучением, смотрит, как мышь проходит выученный путь, приближает и сдвигает изображение лабиринта (рисунок 1). Построение лабиринта, задание параметров и «Обучить N» включают проверку ввода. Второй актёр — мышь: она выбирает направление, получает награду от среды и обновляет Q-таблицу.

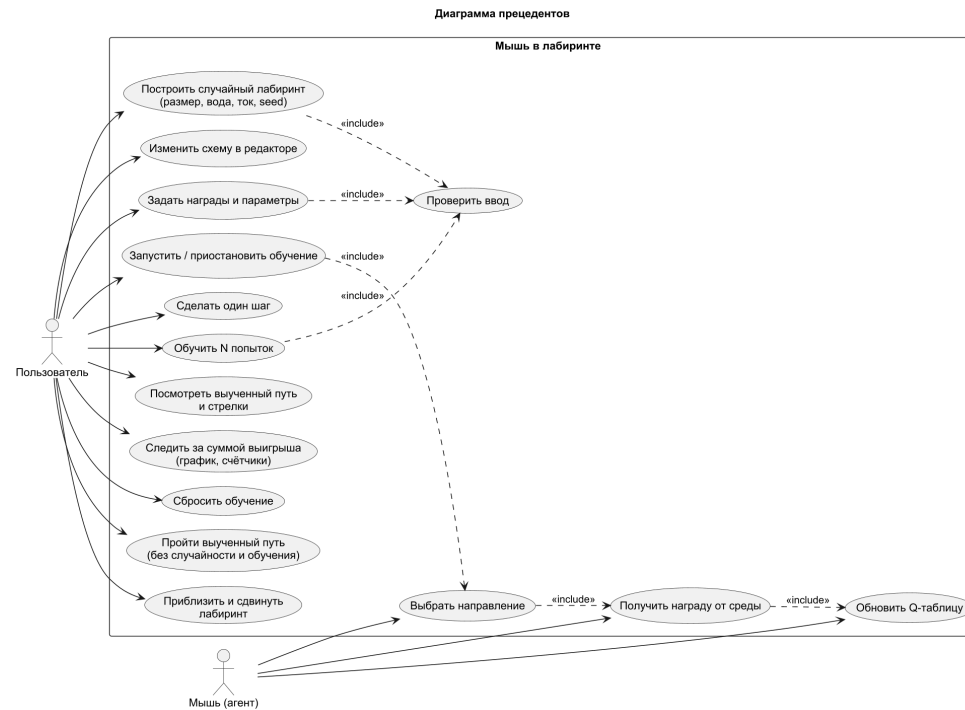


Рисунок 1 — Диаграмма прецедентов

3.2 Диаграммы деятельности

На рисунке 2 показана обработка клика в редакторе: пиксели переводятся в клетку или стену, выбранный инструмент применяется, недопустимые действия отклоняются с объяснением; после изменения обучение начинается заново.

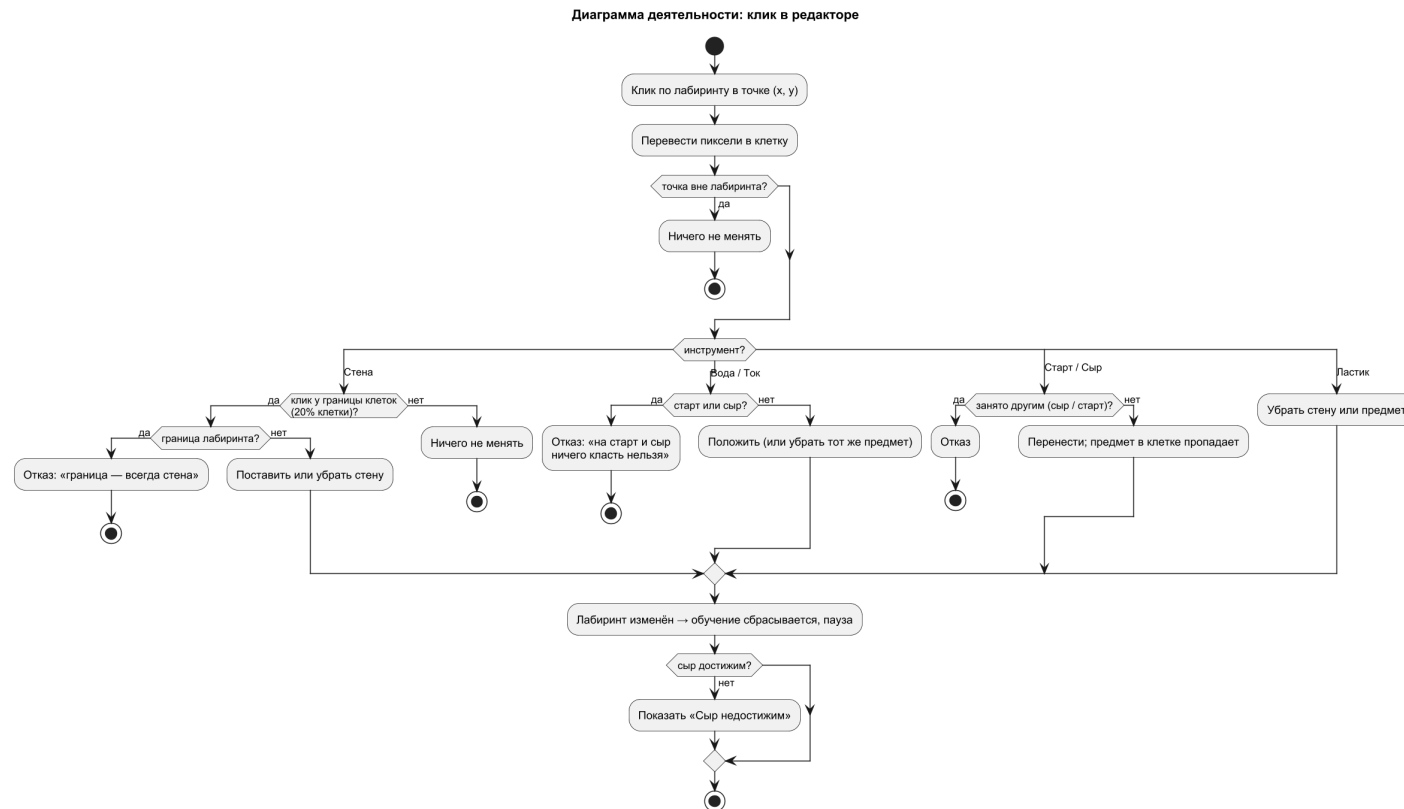


Рисунок 2 — Диаграмма деятельности: клик в редакторе

На рисунке 3 показана одна попытка мыши: выбор направления, ответ среды, обновление Q-таблицы и проверка автостопа после окончания попытки.

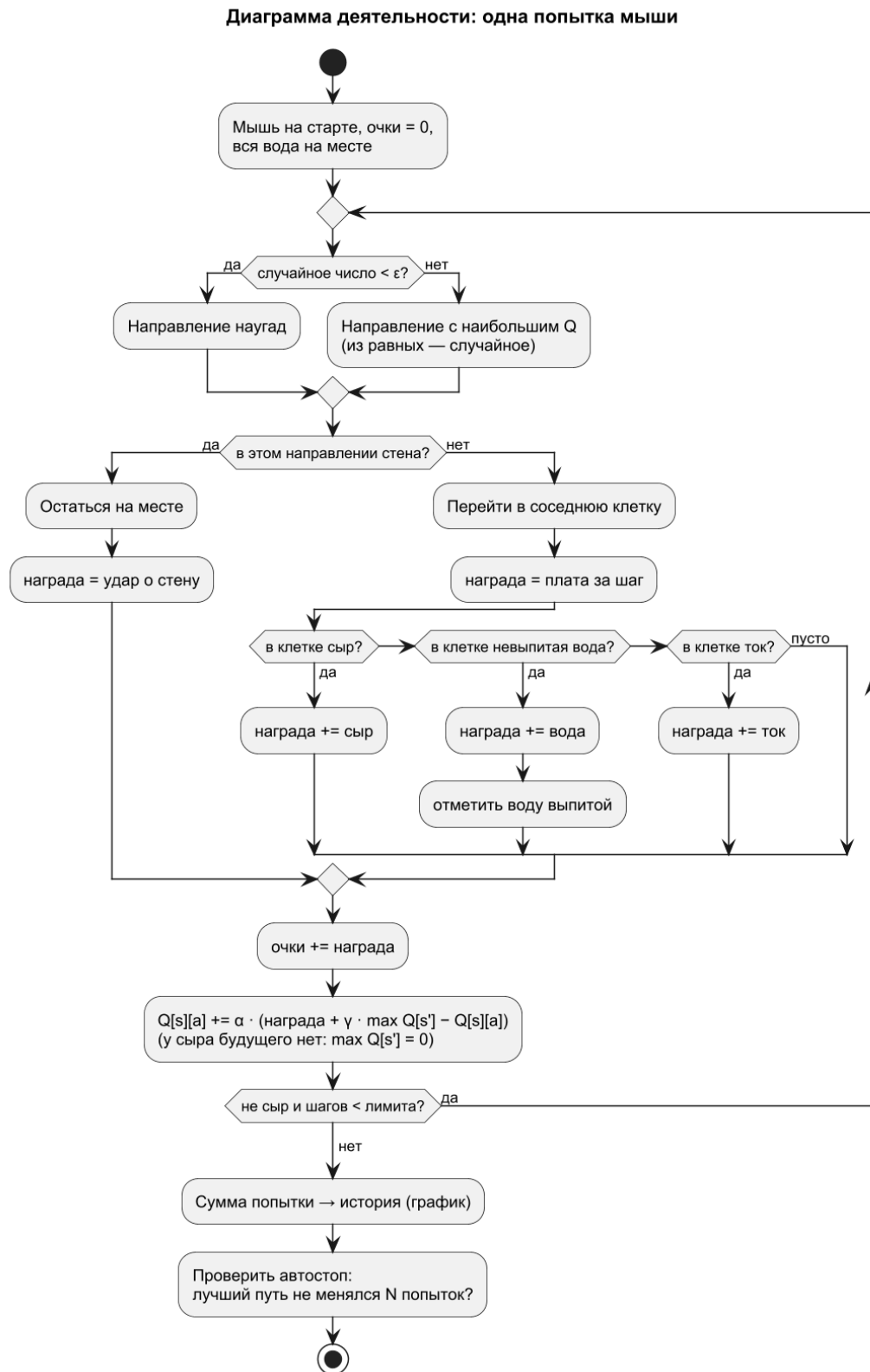


Рисунок 3 — Диаграмма деятельности: одна попытка мыши

3.3 Диаграммы классов

Классы модели показаны на рисунке 4. Maze хранит лабиринт, MazeGenerator строит его по заказу MazeSpec. Attempt — среда: одна попытка, в которой считаются награды и помнится выпитая вода; сам лабиринт при этом не меняется. QTable хранит числа на стрелках, Mouse — агент, выбирающий шаг и обучающийся, Trainer проводит попытки одну за другой, хранит историю сумм и решает, выучен ли путь. Параметры собраны в неизменяемой записи Settings, которая проверяет значения при создании и сообщает обо всех ошибках сразу (InvalidSettingsException).

Диаграмма классов: модель (пакет maze.model)

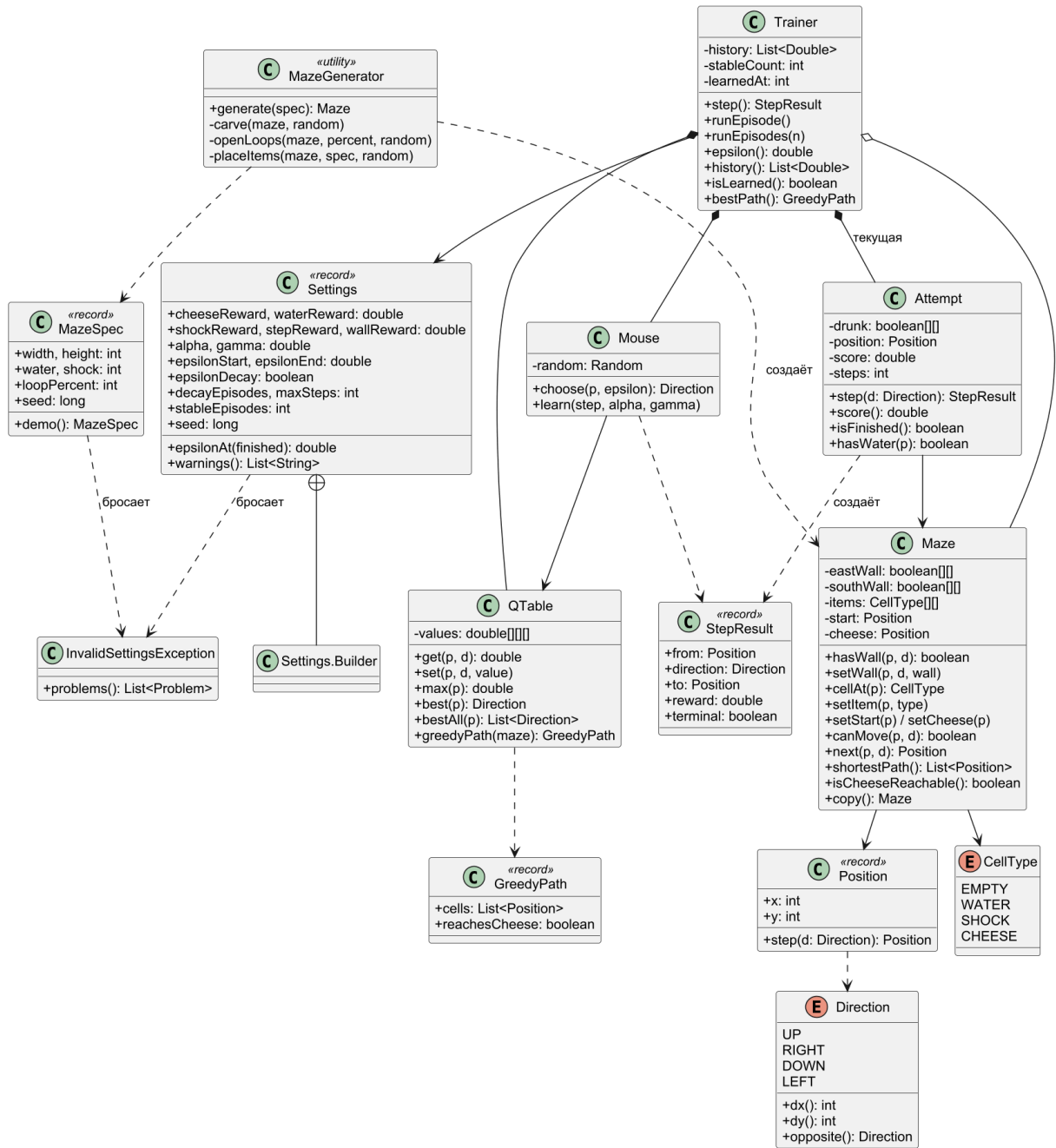


Рисунок 4 — Диаграмма классов модели (пакет maze.model)

Классы управления и окна показаны на рисунке 5. Логика кнопок, редактора, форм и масштаба (класс Viewport) вынесена в пакет maze.controller и не зависит от JavaFX; окно (пакет maze.view) только передаёт действия пользователя контроллеру и рисует состояние.

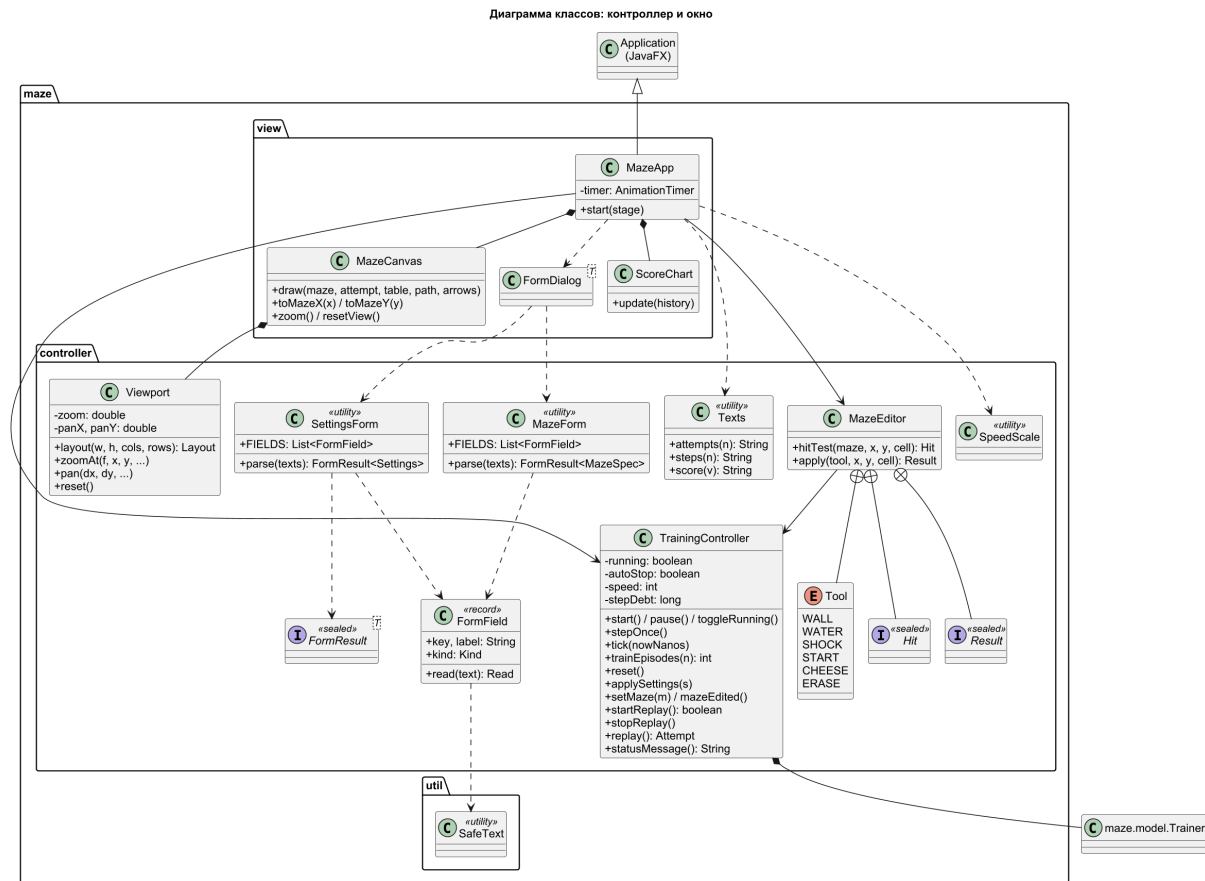


Рисунок 5 — Диаграмма классов управления и окна

3.4 Диаграмма взаимодействия

На рисунке 6 показано взаимодействие объектов за один кадр окна при обучении: таймер передаёт время контроллеру, контроллер выполняет нужное число шагов, на каждом шаге мышь выбирает направление по Q-таблице, среда отвечает наградой, мышь обновляет таблицу; после окончания попытки начинается новая. Затем окно перерисовывает лабиринт и график.

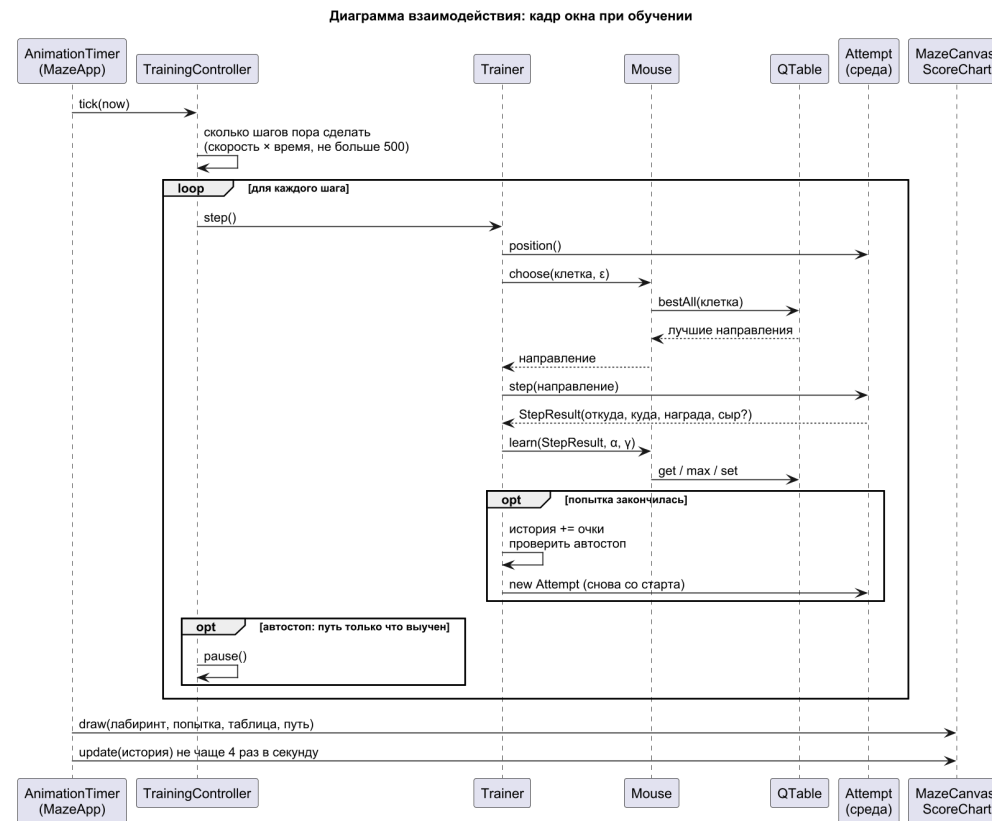


Рисунок 6 — Диаграмма последовательности: кадр окна при обучении

3.5 Диаграммы состояний

На рисунке 7 показаны состояния мыши в течение одной попытки и награды, получаемые при переходах; на рисунке 8 — состояния окна: пауза, обучение, «выучила маршрут» и показ выученного пути.

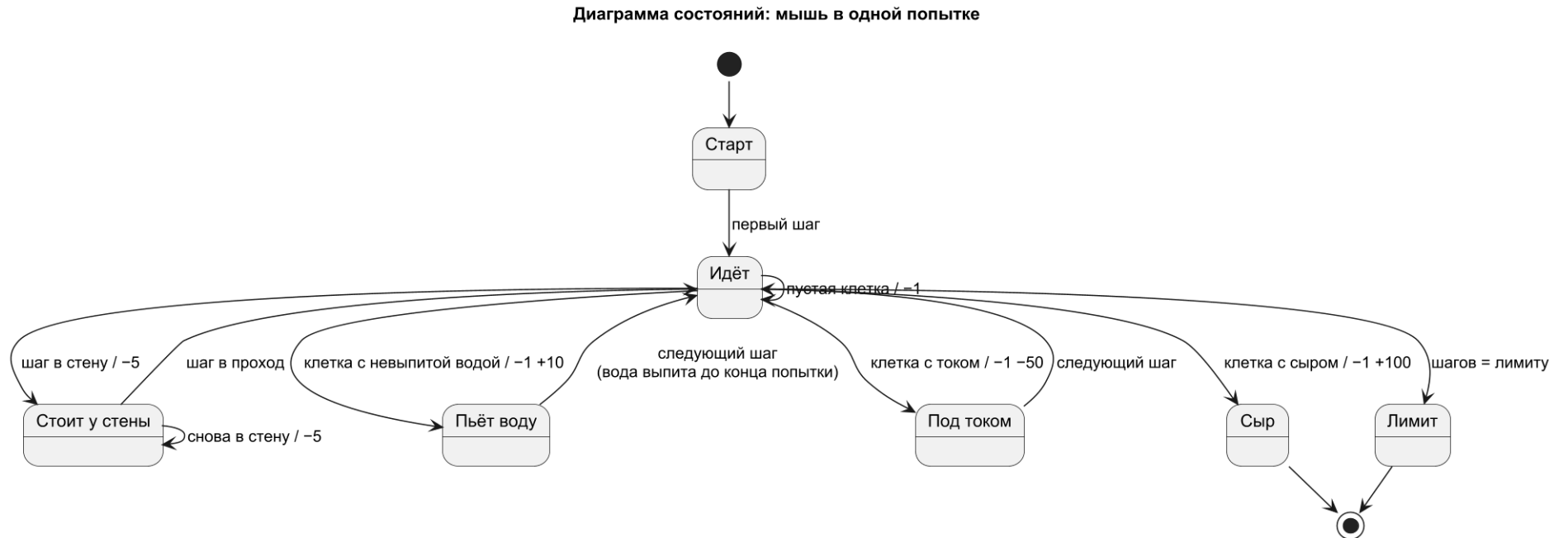


Рисунок 7 — Диаграмма состояний: мышь в одной попытке

Диаграмма состояний: обучение в окне

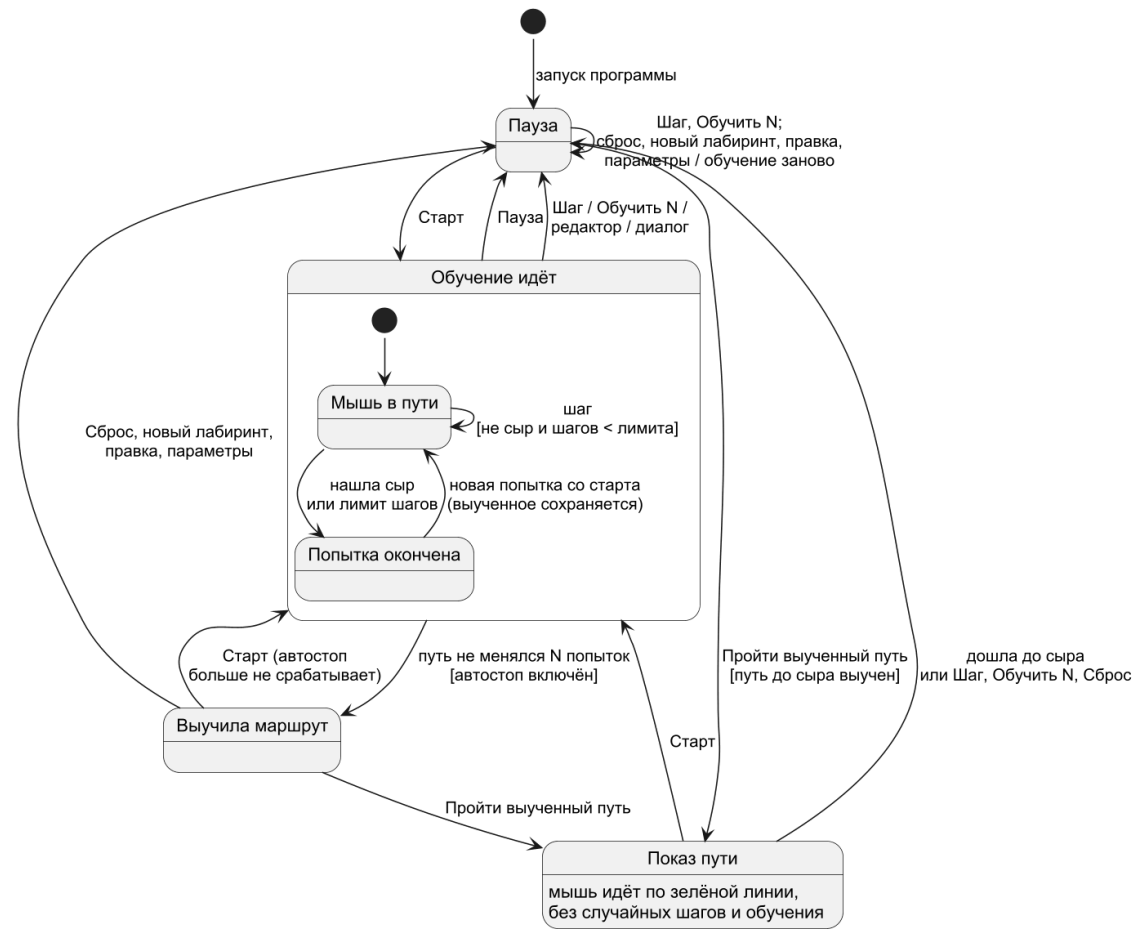


Рисунок 8 — Диаграмма состояний: обучение в окне

3.6 Диаграмма компонентов

Программа оформлена как модуль Java maze и разделена на пакеты по схеме «модель — представление — контроллер» (рисунок 9). Модель не зависит от интерфейса; окно зависит от библиотеки JavaFX.

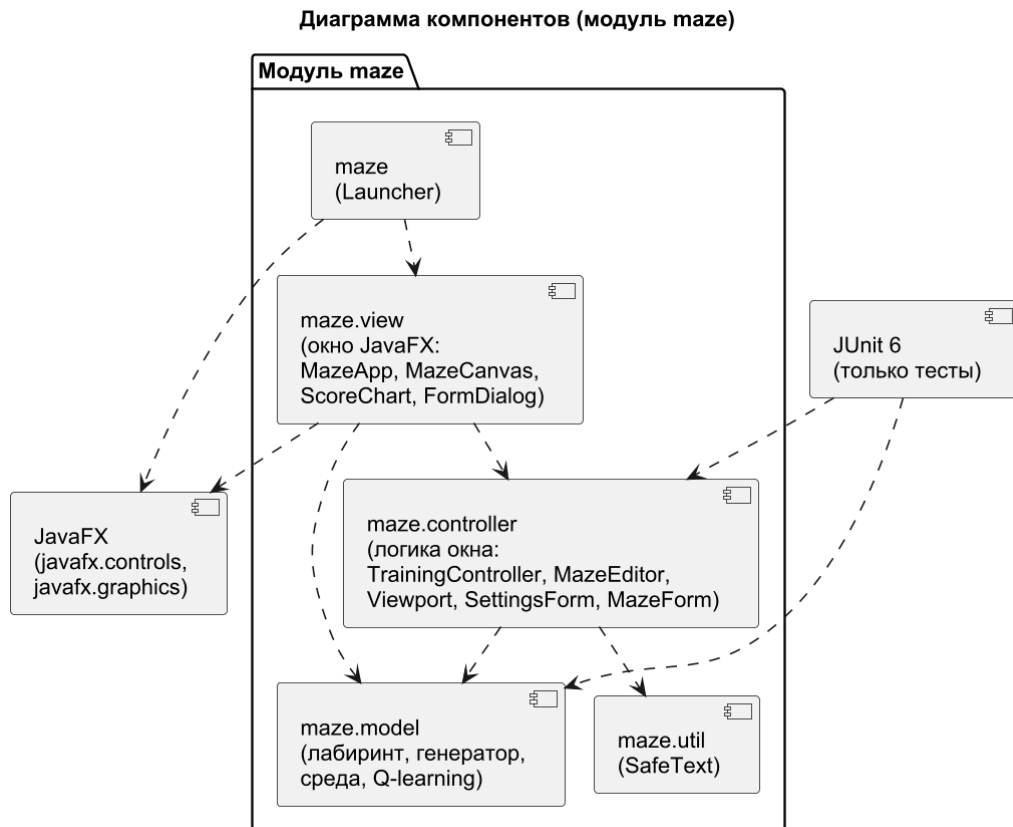


Рисунок 9 — Диаграмма компонентов

4 Реализация

4.1 Средства разработки

- язык Java 25 (JDK Temurin 25);
- библиотека пользовательского интерфейса JavaFX 25.0.4;
- система сборки Maven, среда разработки IntelliJ IDEA;
- система контроля версий Git.

4.2 Структура программы

Таблица 4 — Пакеты программы

Пакет	Назначение
maze	Launcher — точка входа
maze.model	Direction, Position, CellType, Maze, MazeSpec, MazeGenerator — лабиринт; Settings, InvalidSettingsException — параметры; Attempt, StepResult, QTable, Mouse, Trainer, GreedyPath — обучение
maze.controller	TrainingController (старт, пауза, шаг, «обучить N», скорость, автостоп, показ выученного пути), MazeEditor (редактор), Viewport (масштаб и сдвиг), SettingsForm, MazeForm, FormField, FormResult (разбор полей окон), SpeedScale, Texts
maze.view	MazeApp (главное окно), MazeCanvas (рисование лабиринта), Icons, ScoreChart (график), FormDialog (окна параметров и нового лабиринта)
maze.util	SafeText — безопасный вывод введённого пользователем текста в сообщениях

4.3 Применение принципов ООП

Таблица 5 — Принципы ООП в программе

Принцип	Реализация
Инкапсуляция	Стены, предметы, старт и сыр лабиринта закрыты (private) и меняются только методами Maze, которые проверяют корректность; выпитую воду помнит Attempt, а не лабиринт; Q-таблица доступна только через методы get, set, max, best; Settings и MazeSpec неизменяемы и проверяют значения при создании
Абстракция	Mouse работает с таблицей и средой через их методы, не зная, как они устроены; результат шага передаётся записью StepResult; результаты клика редактора и разбора формы описаны закрытыми (sealed) иерархиями Hit, Result, FormResult

Принцип	Реализация
Наследование	MazeApp наследует Application, MazeCanvas — Pane, ScoreChart — LineChart, FormDialog — Dialog, InvalidSettingsException — IllegalArgumentException
Полиморфизм	Окно работает с результатами редактора и форм через общий тип и выбирает действие сопоставлением с образцом (switch по sealed-типам); обобщённый FormDialog<T> один обслуживает и окно параметров, и окно нового лабиринта; JavaFX вызывает переопределённые методы start, layoutChildren

4.4 Оконное приложение

Главное окно (класс MazeApp) содержит лабиринт в центре, панель управления справа и строку кнопок сверху (таблица 6). Лабиринт рисуется на холсте: клетки с водой, током и сыром подкрашены и помечены значками, стены — тонкие линии, мышь — круг с ушами, старт — пунктирное кольцо. Выученный путь показывается зелёной линией, стрелки лучших направлений — по флажку. Выпитая в текущей попытке вода бледнеет. Большой лабиринт можно приближать (до 1600 %) и сдвигать жестами тачпада; при приближении точка под пальцами остаётся на месте.

Таблица 6 — Элементы управления

Элемент	Действие
Старт / Пауза	мышь бежит попытку за попыткой с выбранной скоростью
Шаг	пауза и ровно один шаг мыши
Обучить N	мгновенно выполнить N попыток (до 100 000; не более ~5 млн шагов за раз)
Сбросить обучение	мышь забывает выученное; лабиринт остаётся
Пройти выученный путь	мышь идёт по выученному пути от старта до сыра — без случайных шагов и без обучения
Скорость	от 1 до 5000 шагов в секунду, логарифмическая шкала
Флажки	выученный путь, стрелки направлений, автостоп
Масштаб	щипок на тачпаде или Cmd + колесо — приблизить; два пальца, колесо или перетаскивание — сдвинуть; «Сбросить масштаб» — весь лабиринт целиком
Новый лабиринт...	размер, число клеток с водой и током, процент обходов, seed
Редактор	инструменты Стена, Вода, Ток, Старт, Сыр, Ластик; изменения сбрасывают обучение
Параметры...	награды и параметры обучения

Цикл обучения реализован таймером JavaFX `AnimationTimer`, который вызывается при каждой перерисовке экрана (около 60 раз в секунду). Контроллер накапливает прошедшее время, умноженное на скорость, и выполняет столько целых шагов, сколько накопилось, но не более 500 за кадр, поэтому окно не зависает; время на паузе не превращается в шаги. Кнопка «Пройти выученный путь» ставит обучение на паузу и проводит мышь по лучшим стрелкам от старта до сыра с выбранной скоростью — так видно, что мышь выучила, без шума случайных шагов. Справа выводятся номер попытки, сумма очков и число шагов текущей попытки, доля случайных шагов, сумма прошлой попытки и график «сумма выигрыша за попытку» (каждая попытка и среднее за 20 попыток).

4.5 Защита от некорректного ввода

Разбор полей окон выполняют классы `SettingsForm` и `MazeForm`. Программа не завершается аварийно ни при каком вводе: пустое поле, буквы, запись с экспонентой ($1e3$), NaN и Infinity, цифры других алфавитов, слишком большие числа и управляющие символы приводят к понятному сообщению с указанием поля; длинный ввод в сообщении обрезается. Дробь можно записывать через точку или запятую. Обо всех ошибках сообщается сразу, неверные поля подсвечиваются, окно не закрывается.

Кроме формата проверяется смысл значений: сыр — самая большая награда, ток не является наградой, $\alpha \in (0; 1]$, γ и доли случайных шагов — в $[0; 1]$, клеток с водой и током не больше, чем свободных клеток. Редактор не позволяет убрать границу лабиринта и положить что-либо на старт или сыр; если после правки сыр стал недостижим, выводится предупреждение. Длительные операции ограничены, чтобы окно не зависало: «Обучить N» на лабиринте 100×100 выполняется за доли секунды.

5 Демонстрация визуализации

На рисунке 10 показано окно при запуске: лабиринт 12×9 , мышь на старте, все числа в таблице нулевые. На рисунке 11 — первая попытка на скорости 20 шагов в секунду: мышь ещё ничего не знает и бродит почти случайно, ударяясь о стены; сумма за 60 шагов — уже -232 .

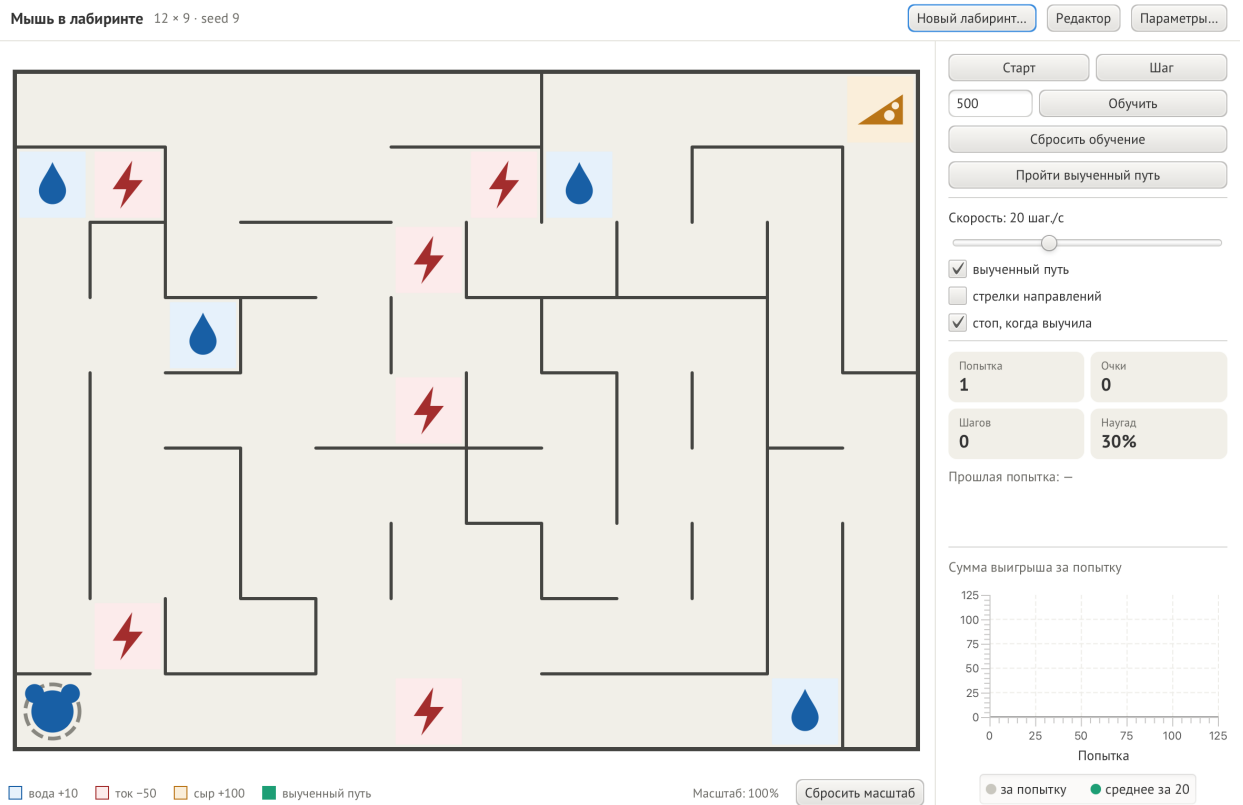


Рисунок 10 — Окно программы при запуске

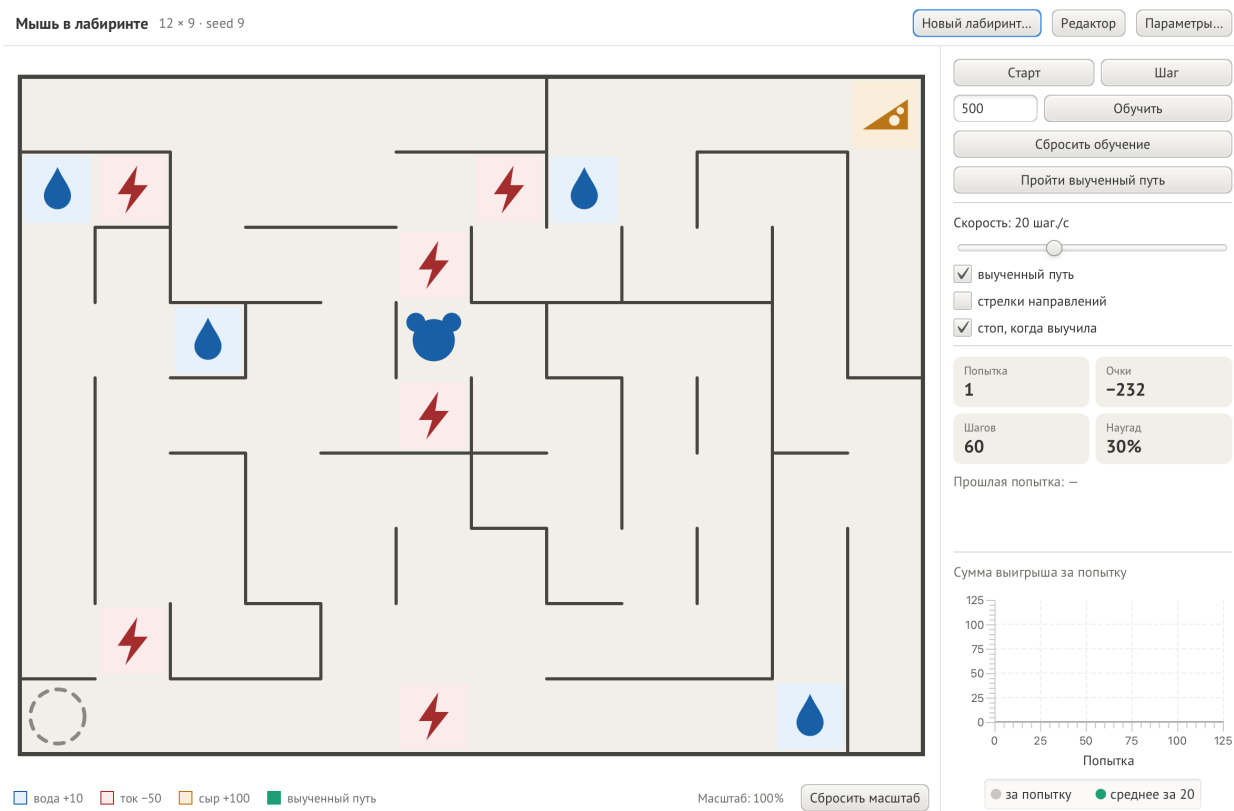
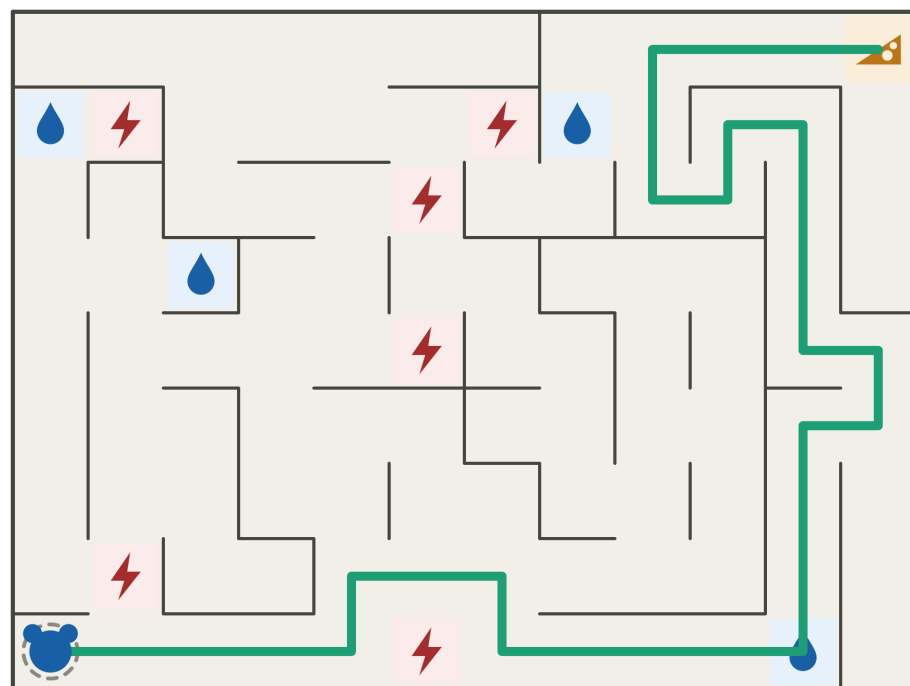


Рисунок 11 — Первая попытка: мышь бродит почти случайно

На рисунке 12 — результат нажатия «Обучить 500»: мышь выучила маршрут за 112 попыток. Кратчайший путь к сыру проходит по нижнему коридору через ток, но выученный путь обходит его сверху и заходит за водой справа. На графике видно, как сумма выигрыша за попытку растёт от -2500 до положительных значений. На рисунке 13 включены стрелки: из каждой клетки показано направление, которое мышь выбрала бы.



вода +10
ток -50
сыр +100
выученный путь

Масштаб: 100%

Сбросить масштаб

Старт Шаг
 500 Обучить
 Сбросить обучение
 Пройти выученный путь

Скорость: 20 шаг./с

☒ выученный путь
☐ стрелки направлений
☒ стоп, когда выучила

Попытка

501

Очки

0

Шагов

0

Наугад

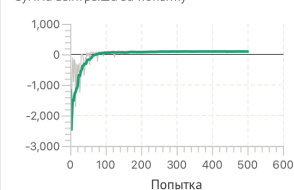
1%

Прошлая попытка: +81

Мышь выучила маршрут за 112 попыток

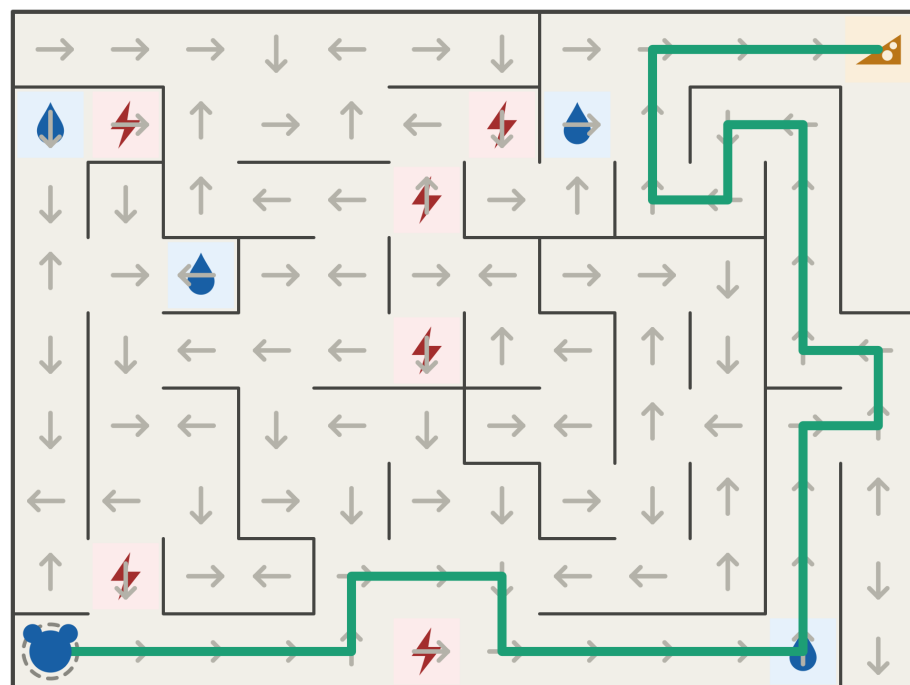
Мышь пробежала ещё 500 попыток

Сумма выигрыша за попытку



за попытку
 среднее за 20

Рисунок 12 — После 500 попыток: выученный путь обходит ток



вода +10
ток -50
сыр +100
выученный путь

Масштаб: 100%

Сбросить масштаб

Старт Шаг
 500 Обучить
 Сбросить обучение
 Пройти выученный путь

Скорость: 20 шаг./с

☒ выученный путь
☒ стрелки направлений
☒ стоп, когда выучила

Попытка

501

Очки

0

Шагов

0

Наугад

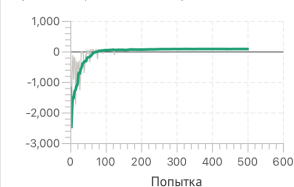
1%

Прошлая попытка: +81

Мышь выучила маршрут за 112 попыток

Мышь пробежала ещё 500 попыток

Сумма выигрыша за попытку



за попытку
 среднее за 20

Рисунок 13 — Стрелки лучших направлений

На рисунке 14 — показ выученного пути: мышь прошла от старта до сыра за 29 шагов и набрала +81 очко, выпитая по дороге вода побледнела. На рисунке 15 показан режим редактора: панель инструментов над лабиринтом; в клетку добавлена вода и поставлена стена, обучение после этого началось заново. На рисунке 16 — лабиринт 30×20 с 10 клетками воды и 25 клетками тока: мышь выучила маршрут за 419 попыток; на рисунке 17 тот же лабиринт приближен в 3 раза у старта.

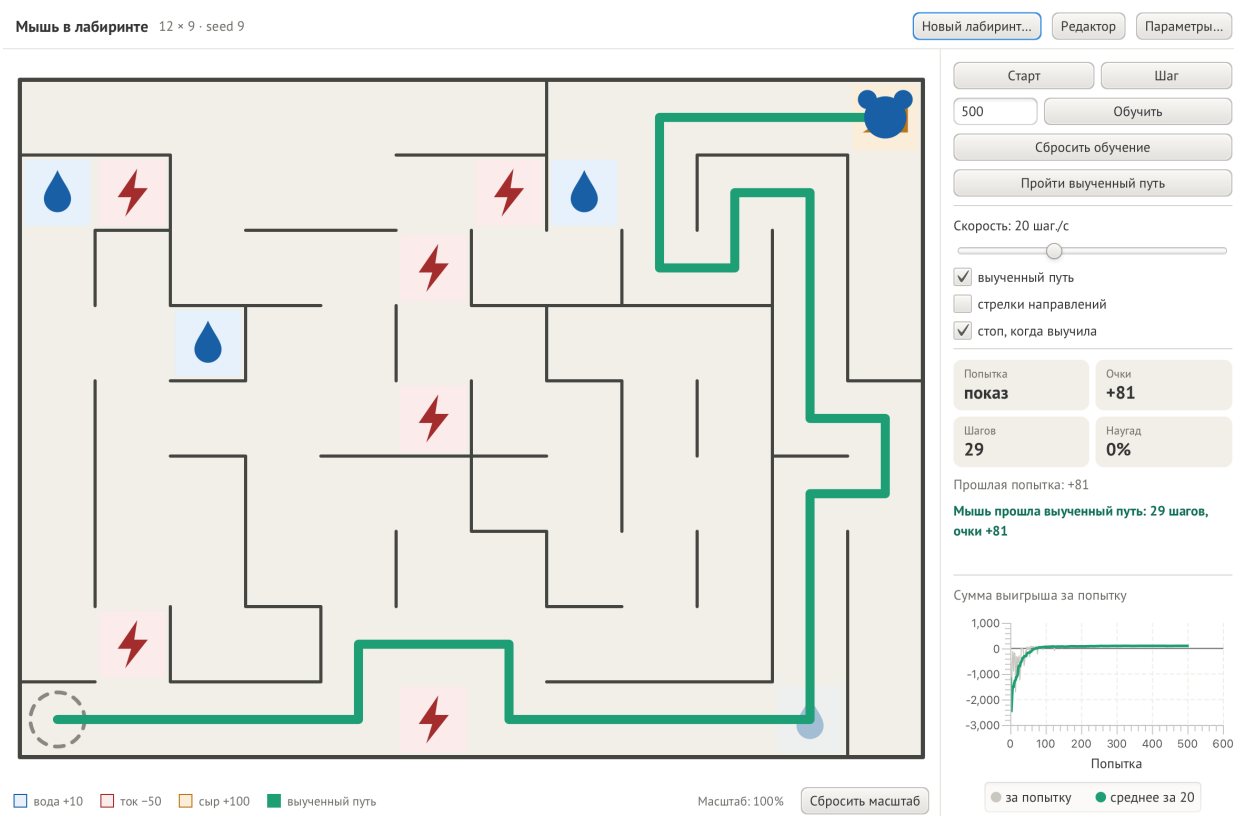


Рисунок 14 — Показ выученного пути

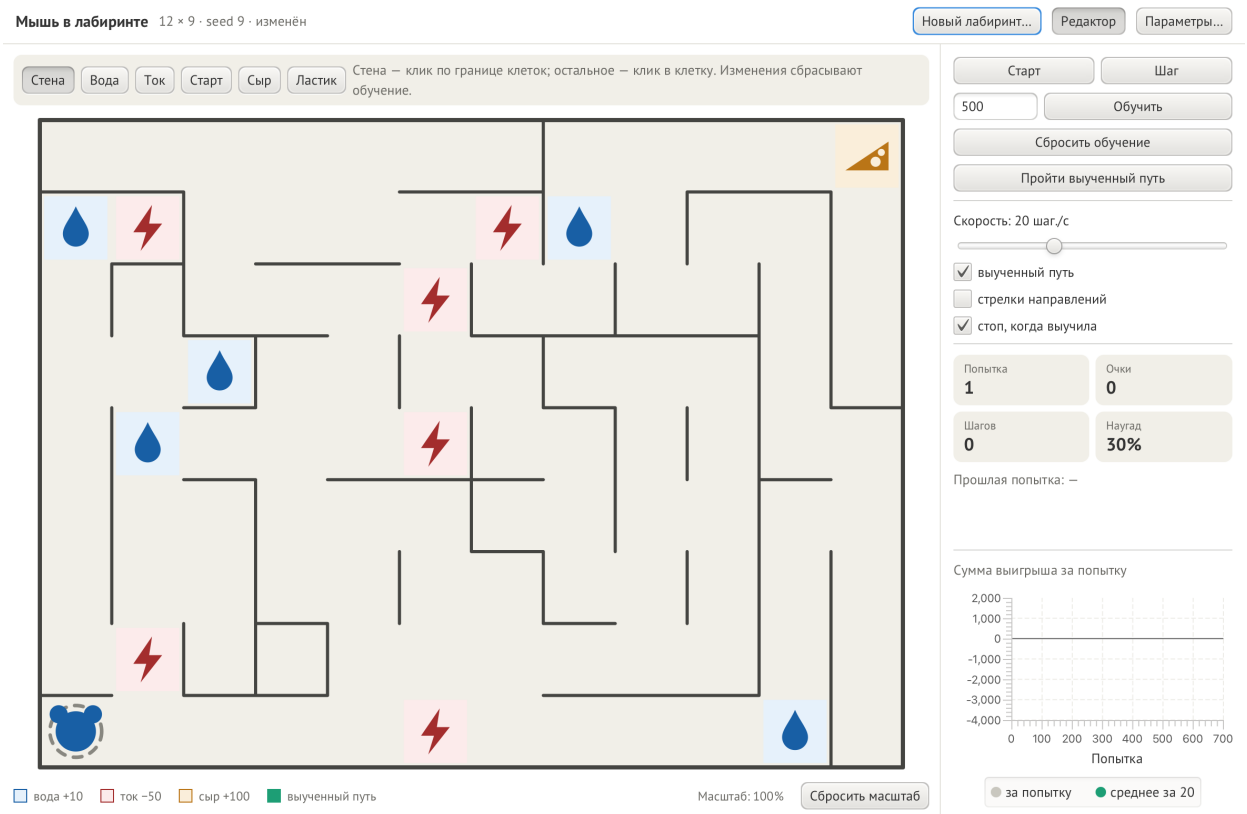


Рисунок 15 — Редактор лабиринта

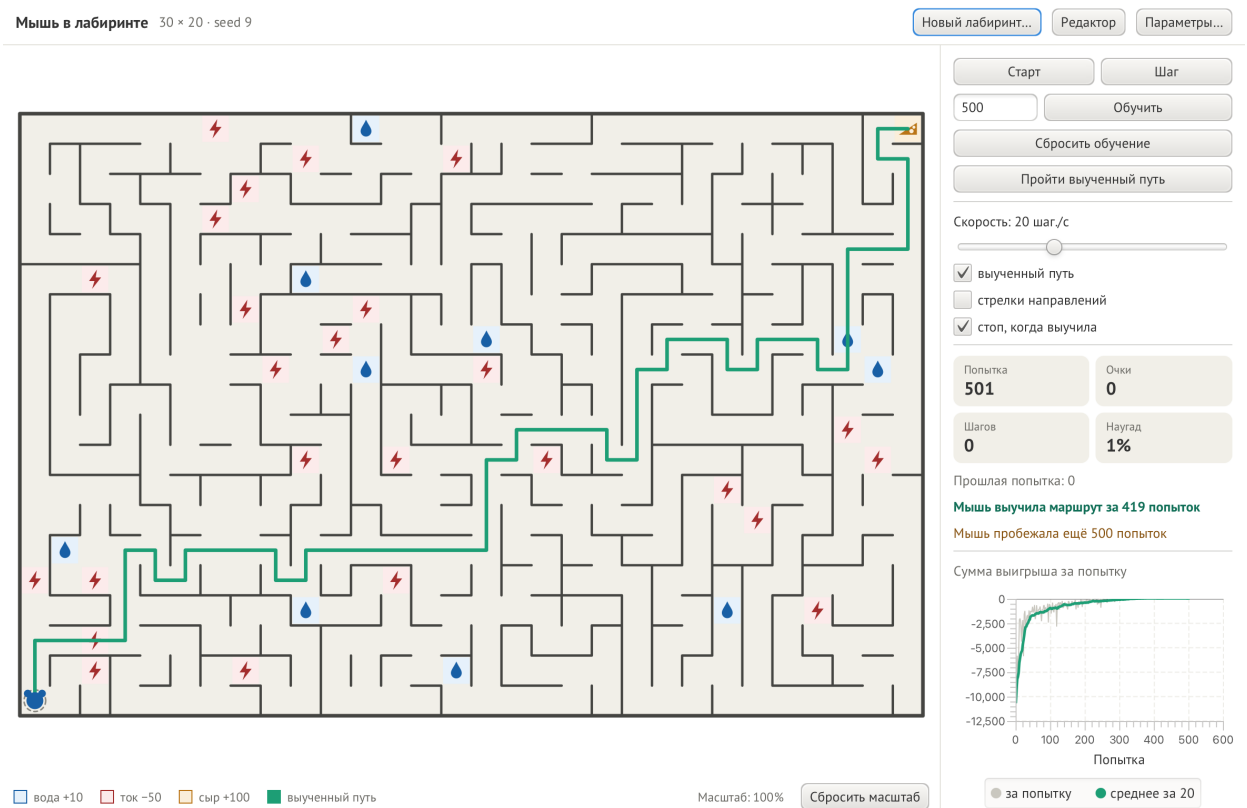


Рисунок 16 — Лабиринт 30×20 после обучения

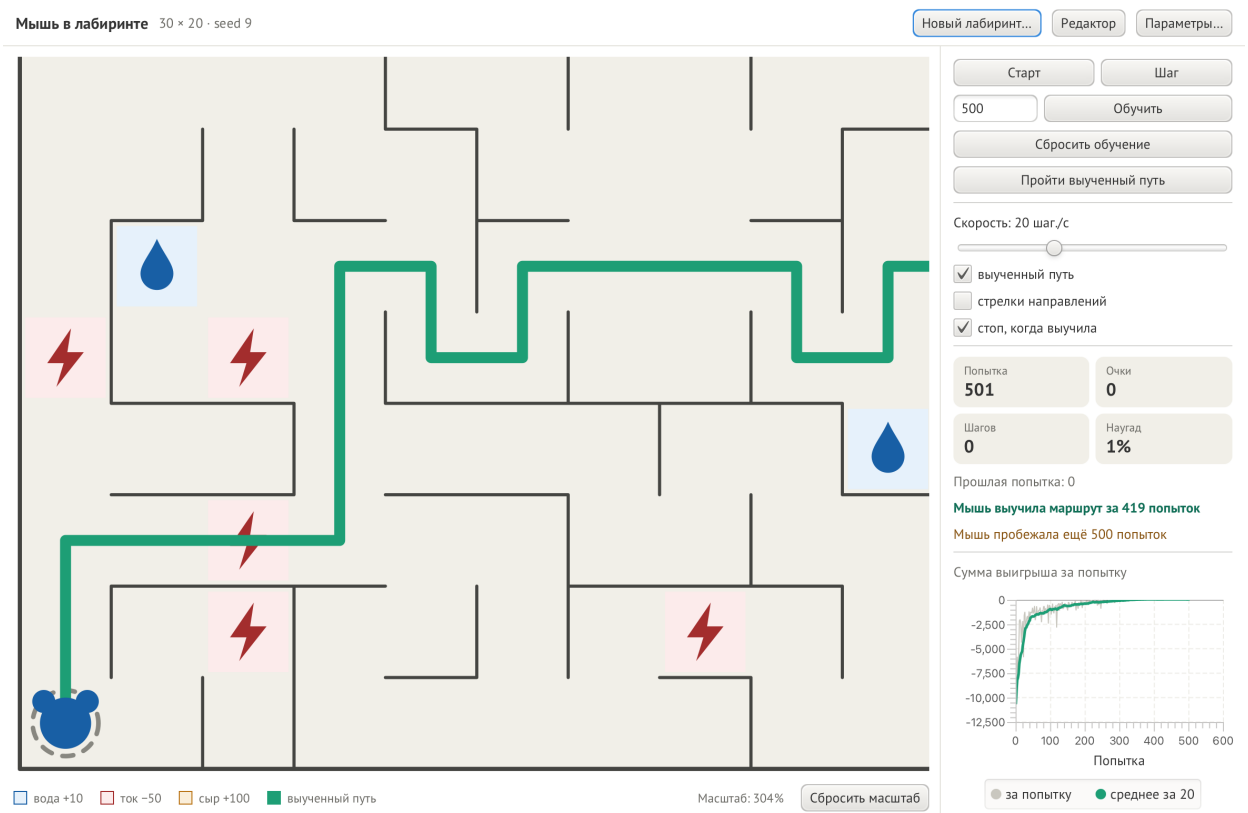


Рисунок 17 — Лабиринт 30×20, приближение 304 %

На рисунке 18 показано окно параметров после ввода ошибочных значений: окно не закрылось, неверные поля подсвечены, под формой перечислены ошибки. На рисунке 19 — предупреждение о почти бесплатных шагах: настройки применяются только по кнопке «Всё равно».

После применения мышь учится заново.

Награды (среда сообщает мыши)

Награда за сыр	100
Награда за воду	10
Награда за удар током	-50
Награда за шаг	-1
Награда за удар о стену	-5

Обучение (Q-learning)

Скорость обучения α	abc
Дисконтирование γ	5
Доля случайных шагов в начале	0.3
Доля случайных шагов в конце	0.01
Доля случайных шагов убывает	☒
За сколько попыток убывает	300

Попытки

Лимит шагов в попытке	2,5
Автостоп: попыток без изменений пути	50
Зерно случайности (seed)	42

Случайный

По умолчанию

- Скорость обучения α : нужно число, а указано "abc"
- Лимит шагов в попытке: нужно целое число, а указано "2,5"

Отмена

Применить

Рисунок 18 — Окно параметров с ошибочным вводом

После применения мышь учится заново.

Награды (среда сообщает мыши)

Награда за сыр	100
Награда за воду	10
Награда за удар током	-50
Награда за шаг	-0,01
Награда за удар о стену	-5

Обучение (Q-learning)

Скорость обучения α	0.2
Дисконтирование γ	1
Доля случайных шагов в начале	0.3
Доля случайных шагов в конце	0.01
Доля случайных шагов убывает	☒
За сколько попыток убывает	300

Попытки

Лимит шагов в попытке	4000
Автостоп: попыток без изменений пути	50
Зерно случайности (seed)	42

Случайный

По умолчанию

Шаги почти бесплатные: мышь может не выучить маршрут. Обычно помогает награда за шаг -0.5 или -1 .

Отмена

Всё равно

Рисунок 19 — Предупреждение о настройках, при которых мышь может не выучить маршрут

Заключение

В ходе лабораторной работы разработана программа «Мышь в лабиринте», в которой агент обучается с подкреплением алгоритмом Q-learning. Мышь не видит карту: на каждом шаге она выбирает направление, среда сообщает награду, и по этой обратной связи мышь выучивает маршрут к сыру, обходя ток и заходя за водой, когда это выгодно. Сумма выигрыша за каждую попытку выводится на экран и на график.

Лабиринт любого размера до 100×100 строится случайно по заданным пользователем размеру, числу клеток с водой и током и проценту обходных путей; схему можно изменить в редакторе. Параметры обучения подобраны перебором: при значениях по умолчанию мышь нашла путь к сыру во всех 60 проверочных лабиринтах размером до 30×30 .

Программа построена на принципах ООП и разделена на модель, контроллер и представление; модель и логика окна не зависят от библиотеки интерфейса. Проект описан UML-диаграммами шести видов. Программа устойчива к некорректному вводу и к попыткам её перегрузить.

Приложение А. Текст программы

Ниже приведён полный текст программы (без автоматических тестов).
Файлы расположены в порядке: сборка и модуль, модель, контроллер, вспомогательный класс, окно. Проект вместе с тестами, диаграммами и этим отчётом опубликован в репозитории <https://github.com/azatmn/mouse-maze>.

Файл pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>ru.spbstu.oop</groupId>
  <artifactId>mouse-maze</artifactId>
  <version>1.0</version>
  <name>Мышь в лабиринте</name>

  <properties>
    <maven.compiler.release>25</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <javafx.version>25.0.4</javafx.version>
    <junit.version>6.1.3</junit.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.openjfx</groupId>
      <artifactId>javafx-controls</artifactId>
      <version>${javafx.version}</version>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>${junit.version}</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.16.0</version>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.6.0</version>
      </plugin>
      <plugin>
        <groupId>org.openjfx</groupId>
        <artifactId>javafx-maven-plugin</artifactId>
        <version>0.0.8</version>
        <configuration>
          <!-- модуль/класс: программа оформлена как модуль Java -->
          <mainClass>maze/maze.Launcher</mainClass>
          <options>
            <!-- разрешить JavaFX загружать системный код без предупреждения -->

```

```

        <option>--enable-native-access=javafx.graphics</option>
    </options>
</configuration>
</plugin>
</plugins>
</build>
</project>

```

Файл `src/main/java/module-info.java`

```

module maze {
    requires javafx.controls;
    exports maze;
    exports maze.model;
    exports maze.controller;
    exports maze.util;
    exports maze.view to javafx.graphics;
    opens maze.view to javafx.graphics;
}

```

Файл `src/main/java/maze/Launcher.java`

```

package maze;

import javafx.application.Application;
import maze.view.MazeApp;

/**
 * Точка входа.
 * Отдельный класс, а не сам MazeApp: если главный класс наследует Application,
 * запуск без модулей JavaFX падает с ошибкой «JavaFX runtime components are missing».
 */
public final class Launcher {

    private Launcher() {

    }

    public static void main(String[] args) {
        Application.launch(MazeApp.class, args);
    }
}

```

Файл `src/main/java/maze/model/Direction.java`

```

package maze.model;

/**
 * Четыре направления хода. Порядок — по часовой стрелке от «вверх»;
 * {@link #ordinal()} служит номером столбца в Q-таблице.
 * Ось y направлена вниз, как в окне.
 */
public enum Direction {
    UP(0, -1),
    RIGHT(1, 0),
    DOWN(0, 1),
    LEFT(-1, 0);

    private final int dx;
    private final int dy;

    Direction(int dx, int dy) {
        this.dx = dx;
        this.dy = dy;
    }

    public int dx() {
        return dx;
    }
}

```

```

    }

    public int dy() {
        return dy;
    }

    public Direction opposite() {
        return values()[ordinal() + 2) % 4];
    }
}

```

Файл src/main/java/maze/model/Position.java

```

package maze.model;

/** Клетка лабиринта: x — столбец, y — строка (сверху вниз). */
public record Position(int x, int y) {

    /** Соседняя клетка в направлении d (без проверки стен и границ). */
    public Position step(Direction d) {
        return new Position(x + d.dx(), y + d.dy());
    }

    @Override
    public String toString() {
        return "(" + x + ", " + y + ")";
    }
}

```

Файл src/main/java/maze/model/CellType.java

```

package maze.model;

/** Что лежит в клетке. Сыр в лабиринте один, его место хранит {@link Maze}. */
public enum CellType {
    EMPTY,
    WATER,
    SHOCK,
    CHEESE
}

```

Файл src/main/java/maze/model/Maze.java

```

package maze.model;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import java.util.Objects;

/**
 * Лабиринт: прямоугольник клеток, тонкие стены между ними, предметы (вода, ток), старт и сыр.
 * Граница лабиринта — всегда стена. Лабиринт меняется только редактором и генератором;
 * мышь его не меняет (выпитую воду помнит попытка, а не лабиринт).
 */
public final class Maze {

    public static final int MAX_SIZE = 100;

    private final int width;
    private final int height;
    /** eastWall[y][x] — стена между (x, y) и (x + 1, y). */
    private final boolean[][] eastWall;

```

```

/** southWall[y][x] – стена между (x, y) и (x, y + 1). */
private final boolean[][] southWall;
private final CellType[][] items;
private Position start;
private Position cheese;

/** Лабиринт без внутренних стен; старт – левый нижний угол, сыр – правый верхний. */
public Maze(int width, int height) {
    checkSize(width, height);
    this.width = width;
    this.height = height;
    this.eastWall = new boolean[height][width];
    this.southWall = new boolean[height][width];
    this.items = new CellType[height][width];
    for (CellType[] row : items) {
        Arrays.fill(row, CellType.EMPTY);
    }
    this.start = new Position(0, height - 1);
    this.cheese = new Position(width - 1, 0);
}

static void checkSize(int width, int height) {
    if (width < 1 || height < 1 || width > MAX_SIZE || height > MAX_SIZE) {
        throw new IllegalArgumentException(
            "размер должен быть от 1 до " + MAX_SIZE + " по каждой стороне, а не "
+ width + "x" + height);
    }
    if (width * height < 2) {
        throw new IllegalArgumentException("нужно хотя бы 2 клетки: для старта и для
сыра");
    }
}

public int width() {
    return width;
}

public int height() {
    return height;
}

public boolean contains(Position p) {
    return p.x() >= 0 && p.y() >= 0 && p.x() < width && p.y() < height;
}

public boolean hasWall(Position p, Direction d) {
    requireInside(p);
    Position n = p.step(d);
    if (!contains(n)) {
        return true;
    }
    return switch (d) {
        case RIGHT -> eastWall[p.y()][p.x()];
        case LEFT -> eastWall[n.y()][n.x()];
        case DOWN -> southWall[p.y()][p.x()];
        case UP -> southWall[n.y()][n.x()];
    };
}

/** Ставит или убирает стену между p и соседом в направлении d. Границу менять нельзя.
*/
public void setWall(Position p, Direction d, boolean wall) {
    requireInside(p);
    Position n = p.step(d);
    if (!contains(n)) {
        throw new IllegalArgumentException("граница лабиринта всегда стена: " + p + " "
+ d);
    }
    switch (d) {
        case RIGHT -> eastWall[p.y()][p.x()] = wall;

```

```

        case LEFT -> eastWall[n.y()][n.x()] = wall;
        case DOWN -> southWall[p.y()][p.x()] = wall;
        case UP -> southWall[n.y()][n.x()] = wall;
    }
}

/** Ставит все внутренние стены. */
public void closeAll() {
    for (int y = 0; y < height; y++) {
        Arrays.fill(eastWall[y], true);
        Arrays.fill(southWall[y], true);
    }
}

public CellType cellAt(Position p) {
    requireInside(p);
    return p.equals(cheese) ? CellType.CHEESE : items[p.y()][p.x()];
}

/** Кладёт в клетку воду, ток или убирает предмет (EMPTY). Старт и сыр заняты. */
public void setItem(Position p, CellType type) {
    Objects.requireNonNull(type, "type");
    requireInside(p);
    if (type == CellType.CHEESE) {
        throw new IllegalArgumentException("сыр переставляется через setCheese");
    }
    if (p.equals(start) || p.equals(cheese)) {
        throw new IllegalArgumentException("на старт и сыр ничего класть нельзя: " +
p);
    }
    items[p.y()][p.x()] = type;
}

public Position start() {
    return start;
}

public Position cheese() {
    return cheese;
}

/** Переносит старт; предмет в новой клетке пропадает. */
public void setStart(Position p) {
    Objects.requireNonNull(p, "start");
    requireInside(p);
    if (p.equals(cheese)) {
        throw new IllegalArgumentException("старт не может совпадать с сыром: " + p);
    }
    items[p.y()][p.x()] = CellType.EMPTY;
    start = p;
}

/** Переносит сыр; предмет в новой клетке пропадает. */
public void setCheese(Position p) {
    Objects.requireNonNull(p, "cheese");
    requireInside(p);
    if (p.equals(start)) {
        throw new IllegalArgumentException("сыр не может совпадать со стартом: " + p);
    }
    items[p.y()][p.x()] = CellType.EMPTY;
    cheese = p;
}

public boolean canMove(Position p, Direction d) {
    return !hasWall(p, d);
}

/** Куда попадёт мышь, шагнув из p в направлении d: соседняя клетка или та же, если там
стена. */

```

```

public Position next(Position p, Direction d) {
    return canMove(p, d) ? p.step(d) : p;
}

/**
 * Кратчайший по числу шагов путь от старта до сыра (поиск в ширину по карте),
 * включая обе клетки. Пустой список – сыр недостижим. Предметы путь не загораживают.
 */
public List<Position> shortestPath() {
    Position[][] from = new Position[height][width];
    ArrayDeque<Position> queue = new ArrayDeque<>();
    from[start.y()][start.x()] = start;
    queue.add(start);
    while (!queue.isEmpty()) {
        Position p = queue.poll();
        for (Direction d : Direction.values()) {
            if (canMove(p, d)) {
                Position n = p.step(d);
                if (from[n.y()][n.x()] == null) {
                    from[n.y()][n.x()] = p;
                    queue.add(n);
                }
            }
        }
    }
    if (from[cheese.y()][cheese.x()] == null) {
        return List.of();
    }
    List<Position> path = new ArrayList<>();
    for (Position p = cheese; !p.equals(start); p = from[p.y()][p.x()]) {
        path.add(p);
    }
    path.add(start);
    Collections.reverse(path);
    return path;
}

public boolean isCheeseReachable() {
    return !shortestPath().isEmpty();
}

public int count(CellType type) {
    int n = 0;
    for (int y = 0; y < height; y++) {
        for (int x = 0; x < width; x++) {
            if (cellAt(new Position(x, y)) == type) {
                n++;
            }
        }
    }
    return n;
}

public Maze copy() {
    Maze c = new Maze(width, height);
    for (int y = 0; y < height; y++) {
        c.eastWall[y] = eastWall[y].clone();
        c.southWall[y] = southWall[y].clone();
        c.items[y] = items[y].clone();
    }
    c.start = start;
    c.cheese = cheese;
    return c;
}

private void requireInside(Position p) {
    Objects.requireNonNull(p, "position");
    if (!contains(p)) {
        throw new IllegalArgumentException("клетка " + p + " вне лабиринта " + width +
            "x" + height);
    }
}

```

```

@Override
public boolean equals(Object o) {
    return o instanceof Maze m
        && width == m.width && height == m.height
        && start.equals(m.start) && cheese.equals(m.cheese)
        && Arrays.deepEquals(eastWall, m.eastWall)
        && Arrays.deepEquals(southWall, m.southWall)
        && Arrays.deepEquals(items, m.items);
}

@Override
public int hashCode() {
    return Objects.hash(width, height, start, cheese,
        Arrays.deepHashCode(eastWall), Arrays.deepHashCode(southWall),
        Arrays.deepHashCode(items));
}
}

```

Файл src/main/java/maze/model/MazeSpec.java

```

package maze.model;

import java.util.ArrayList;
import java.util.List;

/**
 * Заказ на случайный лабиринт. Проверяется при создании; обо всех проблемах сообщает сразу.
 *
 * @param water        сколько клеток с водой
 * @param shock        сколько клеток с током
 * @param loopPercent  какую долю стен, оставшихся после построения «идеального» лабиринта,
 * убрать (0..100):
 *
 * @param seed        0 — к любой клетке ровно один путь, 100 — стен внутри нет
 * зерно генератора: одинаковый seed — одинаковый лабиринт
 */
public record MazeSpec(int width, int height, int water, int shock, int loopPercent, long seed) {

    /**
     * Лабиринт при запуске программы: подобран так, чтобы кратчайший путь к сыру шёл через молнию,
     * а выученный — обходил её и заходил за водой (см. DefaultsTest).
     */
    public static MazeSpec demo() {
        return new MazeSpec(12, 9, 4, 6, 20, 9L);
    }

    public MazeSpec {
        List<InvalidSettingsException.Problem> problems = new ArrayList<>();
        boolean widthOk = width >= 1 && width <= Maze.MAX_SIZE;
        boolean heightOk = height >= 1 && height <= Maze.MAX_SIZE;
        if (!widthOk) {
            problems.add(new InvalidSettingsException.Problem("width",
                "Ширина лабиринта: допустимо от 1 до " + Maze.MAX_SIZE + ", указано " + width));
        }
        if (!heightOk) {
            problems.add(new InvalidSettingsException.Problem("height",
                "Высота лабиринта: допустимо от 1 до " + Maze.MAX_SIZE + ", указано " + height));
        }
        if (widthOk && heightOk && width * height < 2) {
            problems.add(new InvalidSettingsException.Problem("width",
                "Нужно хотя бы 2 клетки: для старта и для сыра"));
        }
        if (water < 0) {
            problems.add(new InvalidSettingsException.Problem("water",
                "Клеток с водой: не может быть меньше 0, указано " + water));
        }
    }
}

```

```

    }
    if (shock < 0) {
        problems.add(new InvalidSettingsException.Problem("shock",
            "Клеток с током: не может быть меньше 0, указано " + shock));
    }
    if (widthOk && heightOk && width * height >= 2 && water >= 0 && shock >= 0) {
        long free = (long) width * height - 2; // без старта и сыра
        long items = (long) water + shock;
        if (items > free) {
            problems.add(new InvalidSettingsException.Problem("shock",
                "Воды и тока " + items + ", а свободных клеток только " + free));
        }
    }
    if (loopPercent < 0 || loopPercent > 100) {
        problems.add(new InvalidSettingsException.Problem("loopPercent",
            "Процент лишних проходов: допустимо от 0 до 100, указано " +
loopPercent));
    }
    if (!problems.isEmpty()) {
        throw new InvalidSettingsException(problems);
    }
}
}

```

Файл src/main/java/maze/model/MazeGenerator.java

```

package maze.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Random;

/**
 * Строит случайный лабиринт по {@link MazeSpec}.
 * <ol>
 * <li>Все стены на месте; обход в глубину со случайным выбором соседа пробивает проходы
 *     получается «идеальный» лабиринт: к любой клетке ровно один путь.</li>
 * <li>Убирается {@code loopPercent}% оставшихся стен — появляются обходные пути,
 *     и мыши есть из чего выбирать.</li>
 * <li>Вода и ток ставятся в случайные клетки, кроме старта и сыра.</li>
 * </ol>
 */
public final class MazeGenerator {

    private MazeGenerator() {
    }

    public static Maze generate(MazeSpec spec) {
        Random random = new Random(spec.seed());
        Maze maze = new Maze(spec.width(), spec.height());
        maze.closeAll();
        carve(maze, random);
        openLoops(maze, spec.loopPercent(), random);
        placeItems(maze, spec, random);
        return maze;
    }

    /** Обход в глубину без рекурсии (стек), чтобы 100x100 не переполнил стек вызовов. */
    private static void carve(Maze maze, Random random) {
        boolean[][] visited = new boolean[maze.height()][maze.width()];
        List<Position> stack = new ArrayList<>();
        Position first = new Position(random.nextInt(maze.width()),
random.nextInt(maze.height()));
        visited[first.y()][first.x()] = true;
        stack.add(first);
        while (!stack.isEmpty()) {
            Position current = stack.getLast();
            List<Direction> options = new ArrayList<>(4);
            for (Direction d : Direction.values()) {

```

```

        Position n = current.step(d);
        if (maze.contains(n) && !visited[n.y()][n.x()]) {
            options.add(d);
        }
    }
    if (options.isEmpty()) {
        stack.removeLast();
        continue;
    }
    Direction d = options.get(random.nextInt(options.size()));
    Position n = current.step(d);
    maze.setWall(current, d, false);
    visited[n.y()][n.x()] = true;
    stack.add(n);
}

}

/** Внутренняя стена: справа или снизу от клетки. */
private record Wall(Position cell, Direction side) {}

private static void openLoops(Maze maze, int percent, Random random) {
    List<Wall> walls = new ArrayList<>();
    for (int y = 0; y < maze.height(); y++) {
        for (int x = 0; x < maze.width(); x++) {
            Position p = new Position(x, y);
            if (x < maze.width() - 1 && maze.hasWall(p, Direction.RIGHT)) {
                walls.add(new Wall(p, Direction.RIGHT));
            }
            if (y < maze.height() - 1 && maze.hasWall(p, Direction.DOWN)) {
                walls.add(new Wall(p, Direction.DOWN));
            }
        }
    }
    Collections.shuffle(walls, random);
    int toOpen = Math.round(walls.size() * percent / 100.0f);
    for (Wall w : walls.subList(0, toOpen)) {
        maze.setWall(w.cell(), w.side(), false);
    }
}

private static void placeItems(Maze maze, MazeSpec spec, Random random) {
    List<Position> free = new ArrayList<>();
    for (int y = 0; y < maze.height(); y++) {
        for (int x = 0; x < maze.width(); x++) {
            Position p = new Position(x, y);
            if (!p.equals(maze.start()) && !p.equals(maze.cheese())) {
                free.add(p);
            }
        }
    }
    Collections.shuffle(free, random);
    for (int i = 0; i < spec.water(); i++) {
        maze.setItem(free.get(i), CellType.WATER);
    }
    for (int i = 0; i < spec.shock(); i++) {
        maze.setItem(free.get(spec.water() + i), CellType.SHOCK);
    }
}
}

```

Файл src/main/java/maze/model/InvalidSettingsException.java

```

package maze.model;

import java.util.List;

/**
 * Настройки некорректны. Содержит сразу все найденные проблемы,
 * чтобы пользователь исправил их за один раз, а не по одной.
 */

```

```

public class InvalidSettingsException extends IllegalArgumentException {

    /** Одна проблема: какое поле и что с ним не так (понятным языком). */
    public record Problem(String field, String message) {
    }

    private final List<Problem> problems;

    public InvalidSettingsException(List<Problem> problems) {
        super("Некорректные настройки: " + String.join("; ",
problems.stream().map(Problem::message).toList()));
        this.problems = List.copyOf(problems);
    }

    public List<Problem> problems() {
        return problems;
    }
}

```

Файл src/main/java/maze/model/Settings.java

```

package maze.model;

import java.util.ArrayList;
import java.util.List;

/**
 * Все параметры обучения в одном месте: награды среды, параметры Q-learning, лимиты.
 * Объект неизменяемый: создаётся через {@link Builder} и проверяется при создании,
 * поэтому некорректные настройки не могут попасть в модель.
 *
 * @param cheeseReward    награда за сыр (+2), больше нуля
 * @param waterReward     награда за воду (+x), меньше награды за сыр
 * @param shockReward     награда за удар током (-y), не больше нуля
 * @param stepReward      плата за каждый шаг, не больше нуля: чтобы короткий путь был
выгоднее длинного
 * @param wallReward      награда за удар о стену (мышь остаётся на месте), не больше нуля
 * @param alpha            $\alpha$  — насколько сильно число сдвигается к новой оценке за один шаг,
(0, 1]
 * @param gamma            $\gamma$  — во сколько раз награда на шаг дальше ценится меньше, [0, 1]
 * @param epsilonStart    доля случайных шагов в начале обучения
 * @param epsilonEnd      доля случайных шагов после {@code decayEpisodes} попыток (если она
убывает)
 * @param epsilonDecay    убывает ли доля случайных шагов; если нет — всегда {@code
epsilonStart}
 * @param decayEpisodes   за сколько попыток доля случайных шагов доходит до {@code
epsilonEnd}
 * @param maxSteps        лимит шагов в одной попытке
 * @param stableEpisodes  сколько попыток подряд лучший путь не должен меняться, чтобы
считать его выученным
 * @param seed            зерно генератора случайных шагов мыши
 */
public record Settings(
    double cheeseReward,
    double waterReward,
    double shockReward,
    double stepReward,
    double wallReward,
    double alpha,
    double gamma,
    double epsilonStart,
    double epsilonEnd,
    boolean epsilonDecay,
    int decayEpisodes,
    int maxSteps,
    int stableEpisodes,
    long seed) {

    public static final double MAX_REWARD = 1_000_000;
    public static final int MAX_COUNT = 1_000_000;
}

```

```

    public Settings {
        Checker check = new Checker();

        boolean cheeseOk = check.range("cheeseReward", "Награда за сыр", cheeseReward, 0,
false, MAX_REWARD);
        boolean waterOk = check.range("waterReward", "Награда за воду", waterReward, 0,
true, MAX_REWARD);
        if (cheeseOk && waterOk && waterReward >= cheeseReward) {
            check.fail("waterReward", "Награда за воду (" + format(waterReward)
+ ") должна быть меньше награды за сыр (" + format(cheeseReward) +
""));
        }
        check.range("shockReward", "Награда за удар током", shockReward, -MAX_REWARD, true,
0);
        boolean stepOk = check.range("stepReward", "Награда за шаг", stepReward, -
MAX_REWARD, true, 0);
        check.range("wallReward", "Награда за удар о стену", wallReward, -MAX_REWARD, true,
0);

        check.range("alpha", "Скорость обучения  $\alpha$ ", alpha, 0, false, 1);
        boolean gammaOk = check.range("gamma", "Коэффициент дисконтирования  $\gamma$ ", gamma, 0,
true, 1);
        if (stepOk && gammaOk && gamma == 1 && stepReward == 0) {
            // без скидки на даль и без платы за шаг длинный путь не хуже короткого:
            // мышь бежит туда-обратно у воды (выпитую воду она не помнит) и не идёт к
сыру
            check.fail("stepReward", "Если  $\gamma = 1$ , награда за шаг должна быть меньше 0, "
+ "иначе мыши незачем торопиться к сыру");
        }
        boolean startOk = check.range("epsilonStart", "Доля случайных шагов в начале",
epsilonStart, 0, true, 1);
        boolean endOk = check.range("epsilonEnd", "Доля случайных шагов в конце",
epsilonEnd, 0, true, 1);
        if (startOk && endOk && epsilonDecay && epsilonEnd > epsilonStart) {
            check.fail("epsilonEnd", "Доля случайных шагов убывает, поэтому в конце (" +
format(epsilonEnd)
+ ") должна быть не больше, чем в начале (" + format(epsilonStart) +
""));
        }

        check.count("decayEpisodes", "Число попыток до конечной доли случайных шагов",
decayEpisodes);
        check.count("maxSteps", "Лимит шагов в попытке", maxSteps);
        check.count("stableEpisodes", "Число попыток без изменений пути для автостопа",
stableEpisodes);

        check.throwIfFailed();
    }

    /**
     * Не ошибка, но мышь при таких настройках может не выучить маршрут. Пороги — из замера
     * на 12 случайных лабиринтах: при бесплатных шагах или при  $\gamma \geq 0.99$  и шаге дешевле 0.1
     * мышь не знает, какая дальняя вода уже выпита, её оценки раздуваются и ничем не
гасятся.
     */
    public List<String> warnings() {
        if (stepReward == 0 || (gamma >= 0.99 && stepReward > -0.1)) {
            return List.of("Шаги почти бесплатные: мышь может не выучить маршрут. "
+ "Обычно помогает награда за шаг -0.5 или -1.");
        }
        return List.of();
    }

    /** Доля случайных шагов в попытке, перед которой закончено {@code finished} попыток.
    */
    public double epsilonAt(int finished) {
        if (!epsilonDecay) {
            return epsilonStart;
        }
    }

```

```

        double progress = Math.min(1.0, (double) finished / decayEpisodes);
        return epsilonStart + (epsilonEnd - epsilonStart) * progress;
    }

    private static String format(double v) {
        return v == Math rint(v) && Math.abs(v) < 1e15 ? String.valueOf((long) v) :
String.valueOf(v);
    }

    /** Копит найденные проблемы, чтобы сообщить обо всех сразу. */
    private static final class Checker {
        private final List<InvalidSettingsException.Problem> problems = new ArrayList<>();

        /** Проверяет, что value в (min, max] или [min, max]. Возвращает true, если всё в
порядке. */
        boolean range(String field, String title, double value, double min, boolean
minIncluded, double max) {
            // NaN не проходит ни одно сравнение, бесконечности — за границами, поэтому
отсекаются здесь же
            boolean aboveMin = minIncluded ? value >= min : value > min;
            if (aboveMin && value <= max) {
                return true;
            }
            String allowed = (minIncluded ? "от " : "больше ") + format(min) + " до " +
format(max);
            String given = Double.isFinite(value) ? format(value) : "не число";
            fail(field, title + ": допустимо " + allowed + ", указано " + given);
            return false;
        }

        void count(String field, String title, int value) {
            if (value < 1 || value > MAX_COUNT) {
                fail(field, title + ": допустимо от 1 до " + MAX_COUNT + ", указано " +
value);
            }
        }

        void fail(String field, String message) {
            problems.add(new InvalidSettingsException.Problem(field, message));
        }

        void throwIfFailed() {
            if (!problems.isEmpty()) {
                throw new InvalidSettingsException(problems);
            }
        }
    }

    public static Settings defaults() {
        return builder().build();
    }

    public static Builder builder() {
        return new Builder();
    }

    /** Билдер с текущими значениями — удобно поменять пару полей у готовых настроек. */
    public Builder toBuilder() {
        return new Builder()

        .cheeseReward(cheeseReward).waterReward(waterReward).shockReward(shockReward)
            .stepReward(stepReward).wallReward(wallReward)
            .alpha(alpha).gamma(gamma)

        .epsilonStart(epsilonStart).epsilonEnd(epsilonEnd).epsilonDecay(epsilonDecay)

        .decayEpisodes(decayEpisodes).maxSteps(maxSteps).stableEpisodes(stableEpisodes)
            .seed(seed);
    }
}

```

```

/** Пошаговая сборка настроек: меняем только нужные поля, остальные берутся по
умолчанию. */
public static final class Builder {
    private double cheeseReward = 100;
    private double waterReward = 10;
    private double shockReward = -50;
    private double stepReward = -1;
    private double wallReward = -5;
    //  $\alpha$ ,  $\gamma$ , доля случайных шагов и лимит подобраны перебором (см. DefaultsTest):
    // на 60 отложенных лабиринтах до 30x30 мышь нашла путь во всех за 1000 попыток
    private double alpha = 0.2;
    private double gamma = 0.995;
    private double epsilonStart = 0.3;
    private double epsilonEnd = 0.01;
    private boolean epsilonDecay = true;
    private int decayEpisodes = 300;
    private int maxSteps = 4000;
    private int stableEpisodes = 50;
    private long seed = 42;

    private Builder() {
    }

    public Builder cheeseReward(double v) {
        cheeseReward = v;
        return this;
    }

    public Builder waterReward(double v) {
        waterReward = v;
        return this;
    }

    public Builder shockReward(double v) {
        shockReward = v;
        return this;
    }

    public Builder stepReward(double v) {
        stepReward = v;
        return this;
    }

    public Builder wallReward(double v) {
        wallReward = v;
        return this;
    }

    public Builder alpha(double v) {
        alpha = v;
        return this;
    }

    public Builder gamma(double v) {
        gamma = v;
        return this;
    }

    public Builder epsilonStart(double v) {
        epsilonStart = v;
        return this;
    }

    public Builder epsilonEnd(double v) {
        epsilonEnd = v;
        return this;
    }
}

```

```

    public Builder epsilonDecay(boolean v) {
        epsilonDecay = v;
        return this;
    }

    public Builder decayEpisodes(int v) {
        decayEpisodes = v;
        return this;
    }

    public Builder maxSteps(int v) {
        maxSteps = v;
        return this;
    }

    public Builder stableEpisodes(int v) {
        stableEpisodes = v;
        return this;
    }

    public Builder seed(long v) {
        seed = v;
        return this;
    }

    public Settings build() {
        return new Settings(cheeseReward, waterReward, shockReward, stepReward,
            wallReward,
            alpha, gamma, epsilonStart, epsilonEnd, epsilonDecay,
            decayEpisodes, maxSteps, stableEpisodes, seed);
    }
}

```

Файл src/main/java/maze/model/StepResult.java

```

package maze.model;

/**
 * Что произошло за один шаг мыши – ровно то, что нужно для обучения.
 */
* @param from        откуда шагнула
* @param direction   куда шагнула
* @param to          где оказалась (та же клетка, если там стена)
* @param reward       сколько получила от среды
* @param terminal     дошла до сыра: у этого шага нет будущего
*/
public record StepResult(Position from, Direction direction, Position to, double reward,
    boolean terminal) {
}

```

Файл src/main/java/maze/model/Attempt.java

```

package maze.model;

import java.util.Objects;

/**
 * Среда: одна попытка мыши от старта до сыра или до лимита шагов.
 * Считает награды и помнит выпитую воду; сам лабиринт не меняет,
 * поэтому в следующей попытке вода снова на месте.
 */
public final class Attempt {

    private final Maze maze;
    private final Settings settings;
}

```

```

private final boolean[][] drunk;
private Position position;
private double score;
private int steps;
private boolean reachedCheese;

public Attempt(Maze maze, Settings settings) {
    this.maze = Objects.requireNonNull(maze, "maze");
    this.settings = Objects.requireNonNull(settings, "settings");
    this.drunk = new boolean[maze.height()][maze.width()];
    this.position = maze.start();
}

public Position position() {
    return position;
}

/** Сумма выигрыша за попытку (условие: мышь должна знать сумму выигрыша или проигрыша). */
public double score() {
    return score;
}

public int steps() {
    return steps;
}

public boolean isFinished() {
    return reachedCheese || steps >= settings.maxSteps();
}

public boolean reachedCheese() {
    return reachedCheese;
}

/** Есть ли в клетке вода, которую ещё не выпили в этой попытке. */
public boolean hasWater(Position p) {
    return maze.cellAt(p) == CellType.WATER && !drunk[p.y()][p.x()];
}

/** Мышь шагает в направлении d, среда отвечает наградой. */
public StepResult step(Direction d) {
    Objects.requireNonNull(d, "direction");
    if (isFinished()) {
        throw new IllegalStateException("попытка уже закончена");
    }
    Position from = position;
    double reward;
    if (!maze.canMove(from, d)) {
        reward = settings.wallReward();
    } else {
        position = from.step(d);
        reward = settings.stepReward() + rewardFor(position);
    }
    steps++;
    score += reward;
    return new StepResult(from, d, position, reward, reachedCheese);
}

private double rewardFor(Position p) {
    return switch (maze.cellAt(p)) {
        case CHEESE -> {
            reachedCheese = true;
            yield settings.cheeseReward();
        }
        case WATER -> {
            if (drunk[p.y()][p.x()]) {
                yield 0;
            }
        }
    };
}

```

```

        drunk[p.y()][p.x()] = true;
        yield settings.waterReward();
    }
    case SHOCK -> settings.shockReward();
    case EMPTY -> 0;
};
    }
}

```

Файл src/main/java/maze/model/GreedyPath.java

```

package maze.model;

import java.util.List;

/**
 * Путь «по лучшим стрелкам» от старта.
 *
 * @param cells пройденные клетки начиная со старта
 * @param reachesCheese дошёл ли путь до сыра (иначе оборвался на петле или стене)
 */
public record GreedyPath(List<Position> cells, boolean reachesCheese) {

    public GreedyPath {
        cells = List.copyOf(cells);
    }
}

```

Файл src/main/java/maze/model/QTable.java

```

package maze.model;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * Q-таблица — «числа на стрелках»: для каждой клетки и каждого из 4 направлений
 * оценка, сколько очков в сумме мышь получит, если шагнёт туда. Вначале все нули.
 */
public final class QTable {

    private final int width;
    private final int height;
    /** values[y][x][направление]. */
    private final double[][][] values;

    public QTable(int width, int height) {
        Maze.checkSize(width, height);
        this.width = width;
        this.height = height;
        this.values = new double[height][width][Direction.values().length];
    }

    public int width() {
        return width;
    }

    public int height() {
        return height;
    }

    public double get(Position p, Direction d) {
        return cell(p)[d.ordinal()];
    }
}

```

```

public void set(Position p, Direction d, double value) {
    cell(p)[d.ordinal()] = value;
}

/** Самое большое число среди 4 стрелок клетки. */
public double max(Position p) {
    double[] q = cell(p);
    double m = q[0];
    for (int i = 1; i < q.length; i++) {
        m = Math.max(m, q[i]);
    }
    return m;
}

/** Направление с самым большим числом; при равенстве — первое по часовой стрелке от
«вверх». */
public Direction best(Position p) {
    return bestAll(p).getFirst();
}

/** Все направления с самым большим числом. */
public List<Direction> bestAll(Position p) {
    double m = max(p);
    List<Direction> best = new ArrayList<>(4);
    for (Direction d : Direction.values()) {
        if (get(p, d) == m) {
            best.add(d);
        }
    }
    return best;
}

/**
 * Путь мыши без случайных шагов: от старта каждый раз по лучшей стрелке.
 * Обрывается, если мышь вернулась в уже пройденную клетку (дальше она ходила бы по
 * кругу).
 */
public GreedyPath greedyPath(Maze maze) {
    if (maze.width() != width || maze.height() != height) {
        throw new IllegalArgumentException("таблица " + width + "x" + height
            + " не подходит к лабиринту " + maze.width() + "x" + maze.height());
    }
    List<Position> cells = new ArrayList<>();
    Set<Position> seen = new HashSet<>();
    Position p = maze.start();
    while (seen.add(p)) {
        cells.add(p);
        if (p.equals(maze.cheese())) {
            return new GreedyPath(cells, true);
        }
        p = maze.next(p, best(p));
    }
    return new GreedyPath(cells, false);
}

private double[] cell(Position p) {
    if (p.x() < 0 || p.y() < 0 || p.x() >= width || p.y() >= height) {
        throw new IllegalArgumentException("клетка " + p + " вне таблицы " + width +
            "x" + height);
    }
    return values[p.y()][p.x()];
}
}

```

Файл src/main/java/maze/model/Mouse.java

```

package maze.model;

import java.util.List;

```

```

import java.util.Objects;
import java.util.Random;

/**
 * Агент Q-learning. Карты не видит: знает только, в какой клетке стоит,
 * и учится по наградам, которые сообщает среда ({@link Attempt}).
 */
public final class Mouse {

    private static final Direction[] DIRECTIONS = Direction.values();

    private final QTable table;
    private final Random random;

    public Mouse(QTable table, Random random) {
        this.table = Objects.requireNonNull(table, "table");
        this.random = Objects.requireNonNull(random, "random");
    }

    /**
     * Выбор шага (ε-жадная стратегия): с вероятностью epsilon — наугад, чтобы пробовать
     * новые пути;
     * иначе — туда, где число больше. Среди равных лучших — случайное, чтобы вначале,
     * когда всё по нулям,
     * мышь не шла всегда вверх.
     */
    public Direction choose(Position p, double epsilon) {
        if (random.nextDouble() < epsilon) {
            return DIRECTIONS[random.nextInt(DIRECTIONS.length)];
        }
        List<Direction> best = table.bestAll(p);
        return best.get(random.nextInt(best.size()));
    }

    /**
     * Обучение после шага — формула Q-learning:
     *  $Q[s][a] = Q[s][a] + \alpha \cdot (r + \gamma \cdot \max_{a'} Q[s'][a'] - Q[s][a])$ .
     * «Награда сейчас + лучшее, что можно получить дальше»; у сыра «дальше» нет.
     */
    public void learn(StepResult step, double alpha, double gamma) {
        double future = step.terminal() ? 0 : table.max(step.to());
        double target = step.reward() + gamma * future;
        double old = table.get(step.from(), step.direction());
        table.set(step.from(), step.direction(), old + alpha * (target - old));
    }
}

```

Файл src/main/java/maze/model/Trainer.java

```

package maze.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Objects;
import java.util.Random;

/**
 * Обучение: попытки одна за другой. Каждый шаг — мышь выбирает направление, среда отвечает
 * наградой,
 * мышь учится. Кончилась попытка — её сумма идёт в историю, мышь снова на старте,
 * выученное остаётся.
 */
public final class Trainer {

    private final Maze maze;
    private final Settings settings;
    private final QTable table;

```

```

private final Mouse mouse;
private final List<Double> history = new ArrayList<>();
private Attempt attempt;
private GreedyPath lastPath;
private int stableCount;
private int learnedAt = -1;
private long totalSteps;

public Trainer(Maze maze, Settings settings) {
    this.maze = Objects.requireNonNull(maze, "maze");
    this.settings = Objects.requireNonNull(settings, "settings");
    this.table = new QTable(maze.width(), maze.height());
    this.mouse = new Mouse(table, new Random(settings.seed()));
    this.attempt = new Attempt(maze, settings);
}

public Maze maze() {
    return maze;
}

public Settings settings() {
    return settings;
}

public QTable table() {
    return table;
}

/** Текущая (ещё не законченная) попытка. */
public Attempt attempt() {
    return attempt;
}

/** Номер текущей попытки, с 1. */
public int episode() {
    return history.size() + 1;
}

public int finishedEpisodes() {
    return history.size();
}

/** Сколько всего шагов сделала мышь за всё обучение. */
public long totalSteps() {
    return totalSteps;
}

/** Доля случайных шагов в текущей попытке. */
public double epsilon() {
    return settings.epsilonAt(history.size());
}

/** Один шаг мыши. Если попытка на нём закончилась — сразу начинается следующая. */
public StepResult step() {
    StepResult result = attempt.step(mouse.choose(attempt.position(), epsilon()));
    totalSteps++;
    mouse.learn(result, settings.alpha(), settings.gamma());
    if (attempt.isFinished()) {
        finishAttempt();
    }
    return result;
}

/** Доводит текущую попытку до конца. */
public void runEpisode() {
    int target = episode() + 1;
    while (episode() < target) {
        step();
    }
}

```

```

    }
}

public void runEpisodes(int count) {
    if (count < 0) {
        throw new IllegalArgumentException("число попыток не может быть отрицательным:
" + count);
    }
    for (int i = 0; i < count; i++) {
        runEpisode();
    }
}

/** Суммы выигрыша законченных попыток по порядку. */
public List<Double> history() {
    return Collections.unmodifiableList(history);
}

/** Сумма последней законченной попытки; NaN, пока ни одной не было. */
public double lastScore() {
    return history.isEmpty() ? Double.NaN : history.getLast();
}

/** Лучший путь не менялся (и доходил до сыра) {@code stableEpisodes} попыток подряд.
*/
public boolean isLearned() {
    return stableCount >= settings.stableEpisodes();
}

/** Номер попытки, после которой путь впервые признан выученным; -1 — ещё не выучен. */
public int learnedAt() {
    return learnedAt;
}

public GreedyPath bestPath() {
    return table.greedyPath(maze);
}

private void finishAttempt() {
    history.add(attempt.score());
    GreedyPath path = bestPath();
    if (!path.reachesCheese()) {
        stableCount = 0;
    } else if (path.equals(lastPath)) {
        stableCount++;
    } else {
        stableCount = 1;
    }
    lastPath = path;
    if (learnedAt < 0 && isLearned()) {
        learnedAt = history.size();
    }
    attempt = new Attempt(maze, settings);
}
}

```

Файл src/main/java/maze/controller/TrainingController.java

```

package maze.controller;

import maze.model.Attempt;
import maze.model.Maze;
import maze.model.Settings;
import maze.model.Trainer;

import java.util.Objects;

/**

```

```

* Управление обучением для окна: старт, пауза, шаг, «обучить N попыток», сброс, скорость,
автостоп.
* Не знает ничего про JavaFX — окно только вызывает методы и рисует состояние,
* поэтому всю логику кнопок можно проверить обычными тестами.
* <p>
* Время приходит снаружи через {@link #tick(long)} (окно вызывает его на каждом кадре),
* а контроллер сам решает, сколько шагов сделать, исходя из скорости.
*/
public class TrainingController {

    public static final int MIN_SPEED = 1;
    public static final int MAX_SPEED = 5000;
    public static final int DEFAULT_SPEED = 20;
    /** Больше шагов за один кадр не делаем, чтобы окно не подвисало. */
    public static final int MAX_STEPS_PER_TICK = 500;
    /** Больше попыток за одно нажатие «Обучить» не делаем. */
    public static final int MAX_EPISODES_AT_ONCE = 100_000;
    /** «Обучить» останавливается, когда шагов набралось столько (иначе большой лабиринт
повесит окно). */
    public static final long STEP_BUDGET = 5_000_000;

    private static final long NANOS_PER_SECOND = 1_000_000_000L;

    private Maze maze;
    private Settings settings;
    private Trainer trainer;

    private boolean running;
    /** Показ выученного пути: отдельная попытка без случайных шагов и без обучения. */
    private Attempt replay;
    private boolean replaying;
    private boolean autoStop = true;
    private int speed = DEFAULT_SPEED;
    /** Время прошлого кадра; пока его нет (сразу после старта), первый кадр только
запоминает время. */
    private boolean hasLastTick;
    private long lastTick;
    /** Накопленное время × скорость: целая часть от деления на секунду — сколько шагов
пора сделать. */
    private long stepDebt;

    public TrainingController(Maze maze, Settings settings) {
        this.maze = Objects.requireNonNull(maze, "maze");
        this.settings = Objects.requireNonNull(settings, "settings");
        reset();
    }

    /** Запустить. Повторное нажатие ничего не меняет. */
    public void start() {
        if (running) {
            return;
        }
        stopReplay();
        running = true;
        restartClock();
    }

    public void pause() {
        running = false;
    }

    /** Одна кнопка «Старт/Пауза». */
    public void toggleRunning() {
        if (running) {
            pause();
        } else {
            start();
        }
    }
}

```

```

public boolean isRunning() {
    return running;
}

/** Кнопка «Шаг»: ставит на паузу и делает ровно один шаг мыши. */
public void stepOnce() {
    pause();
    stopReplay();
    trainer.step();
}

/** Очередной кадр окна. nowNanos — текущее время в наносекундах. */
public void tick(long nowNanos) {
    if (!running && !replaying) {
        return;
    }
    long due = dueSteps(nowNanos);
    for (int i = 0; i < due; i++) {
        if (replaying) {
            replay.step(trainer.table().best(replay.position()));
            if (replay.isFinished()) {
                replaying = false;
                return;
            }
            continue;
        }
        boolean wasLearned = trainer.learnedAt() >= 0;
        trainer.step();
        if (autoStop && !wasLearned && trainer.learnedAt() >= 0) {
            pause();
            return;
        }
    }
}

/** Сколько шагов пора сделать к этому кадру (скорость * прошедшее время, не больше лимита за кадр). */
private long dueSteps(long nowNanos) {
    if (!hasLastTick || nowNanos < lastTick) {
        hasLastTick = true;
        lastTick = nowNanos;
        return 0;
    }
    // Больше секунды за кадр не учитываем: иначе после долгого «замерзания» набегит
    // гигантский долг
    long elapsed = Math.min(nowNanos - lastTick, NANOS_PER_SECOND);
    lastTick = nowNanos;
    stepDebt += elapsed * speed;
    long due = stepDebt / NANOS_PER_SECOND;
    stepDebt %= NANOS_PER_SECOND;
    // не успеваем — не догоняем, просто идём медленнее
    return Math.min(due, MAX_STEPS_PER_TICK);
}

/** Время, проведённое на паузе, не должно превратиться в шаги. */
private void restartClock() {
    hasLastTick = false;
    stepDebt = 0;
}

/** Скорость в шагах в секунду; значения вне допустимого диапазона прижимаются к границе. */
public void setSpeed(int stepsPerSecond) {
    speed = Math.clamp(stepsPerSecond, MIN_SPEED, MAX_SPEED);
}

public int getSpeed() {
    return speed;
}

```

```

    /** Останавливаться ли, когда мышь выучила маршрут (срабатывает один раз за обучение).
    */
    public void setAutoStop(boolean autoStop) {
        this.autoStop = autoStop;
    }

    public boolean isAutoStop() {
        return autoStop;
    }

    /**
     * Кнопка «Обучить N попыток»: ставит на паузу и мгновенно прогоняет попытки.
     * Не больше {@link #MAX_EPISODES_AT_ONCE} попыток и примерно {@link #STEP_BUDGET}
    шагов:
     * бюджет проверяется между попытками, поэтому попытка не обрывается посередине.
     *
     * @return сколько попыток сделано на самом деле
     */
    public int trainEpisodes(int count) {
        pause();
        stopReplay();
        int target = Math.min(Math.max(count, 0), MAX_EPISODES_AT_ONCE);
        long start = trainer.totalSteps();
        int done = 0;
        // текущая попытка могла начаться кнопкой «Шаг»: доводим её до конца, это тоже одна
    из N
        while (done < target && trainer.totalSteps() - start < STEP_BUDGET) {
            trainer.runEpisode();
            done++;
        }
        return done;
    }

    /** Кнопка «Сброс»: мышь забывает всё выученное; лабиринт и настройки те же. */
    public void reset() {
        pause();
        stopReplay();
        trainer = new Trainer(maze, settings);
    }

    /** Новые настройки из окна параметров: обучение начинается заново. */
    public void applySettings(Settings newSettings) {
        settings = Objects.requireNonNull(newSettings, "settings");
        reset();
    }

    /** Новый лабиринт (сгенерированный): обучение начинается заново. */
    public void setMaze(Maze newMaze) {
        maze = Objects.requireNonNull(newMaze, "maze");
        reset();
    }

    /** Лабиринт поменяли в редакторе: выученное больше не соответствует ему. */
    public void mazeEdited() {
        reset();
    }

    /**
     * Кнопка «Пройти выученный путь»: обучение на паузе, мышь идёт от старта по лучшим
    стрелкам —
     * без случайных шагов и ничего не выучивая. Шагает по tick со скоростью слайдера.
     *
     * @return false, если выученного пути до сыра пока нет
     */
    public boolean startReplay() {
        pause();
        stopReplay();
        if (!trainer.bestPath().reachesCheese()) {
            return false;
        }
    }

```

```

    }
    replay = new Attempt(maze, settings);
    replaying = true;
    restartClock();
    return true;
}

/** Мышь сейчас идёт по выученному пути. */
public boolean isReplaying() {
    return replaying;
}

/** Попытка показа (идёт или уже закончилась) или null, если показа нет. */
public Attempt replay() {
    return replay;
}

public void stopReplay() {
    replay = null;
    replaying = false;
}

public Maze maze() {
    return maze;
}

public Settings settings() {
    return settings;
}

public Trainer trainer() {
    return trainer;
}

/** Что сообщить пользователю: сыр недостижим, мышь выучила маршрут или ничего. */
public String statusMessage() {
    if (replay != null) {
        return replaying
            ? "Мышь идёт по выученному пути: шаг " + replay.steps()
            : "Мышь прошла выученный путь: " + Texts.steps(replay.steps()) + ",
очки " + Texts.score(replay.score());
    }
    if (!maze.isCheeseReachable()) {
        return "Сыр недостижим: мышь не сможет его найти";
    }
    if (trainer.learnedAt() >= 0) {
        return "Мышь выучила маршрут за " + Texts.attempts(trainer.learnedAt());
    }
    return "";
}
}

```

Файл src/main/java/maze/controller/MazeEditor.java

```

package maze.controller;

import maze.model.CellType;
import maze.model.Direction;
import maze.model.Maze;
import maze.model.Position;

import java.util.Objects;

/**
 * Редактор лабиринта без JavaFX: клик в точку холста → какая это клетка или стенка → что с
ней сделать.
 * Меняет текущий лабиринт контроллера; после любого изменения обучение начинается заново,
 * потому что выученное относится к старому лабиринту.

```

```

*/
public final class MazeEditor {

    /** Доля размера клетки у её границы, где клик считается кликом по стене. */
    public static final double EDGE_ZONE = 0.2;

    /** Инструмент, выбранный в панели редактора. */
    public enum Tool {
        WALL,
        WATER,
        SHOCK,
        START,
        CHEESE,
        ERASE
    }

    /** Куда попал клик. */
    public sealed interface Hit {
        /** Внутрь клетки. */
        record Cell(Position cell) implements Hit {
        }

        /**
         * На стену. Внутренняя стена всегда записана как правая или нижняя у левой/верхней
         клетки,
         * чтобы одна и та же стена не имела двух записей; внешняя граница — как сторона
         крайней клетки.
         */
        record Edge(Position cell, Direction side) implements Hit {
        }

        /** Мимо лабиринта. */
        record Outside() implements Hit {
        }
    }

    /** Что вышло из клика. */
    public sealed interface Result {
        record Changed() implements Result {
        }

        record Unchanged() implements Result {
        }

        /** Так нельзя; message — объяснение для пользователя. */
        record Rejected(String message) implements Result {
        }
    }

    private final TrainingController controller;

    public MazeEditor(TrainingController controller) {
        this.controller = Objects.requireNonNull(controller, "controller");
    }

    /**
     * Куда попадает точка (px, py) холста, если клетка на экране — квадрат cellSize
     пикселей,
     * а левый верхний угол лабиринта — (0, 0).
     */
    public static Hit hitTest(Maze maze, double px, double py, double cellSize) {
        Position cell = cellAt(maze, px, py, cellSize);
        if (cell == null) {
            return new Hit.Outside();
        }
        double inX = px - cell.x() * cellSize; // расстояние от левой границы клетки
        double inY = py - cell.y() * cellSize;
        // до ближайшей вертикальной и горизонтальной границы

```

```

        double dx = Math.min(inX, cellSize - inX);
        double dy = Math.min(inY, cellSize - inY);
        if (Math.min(dx, dy) > cellSize * EDGE_ZONE) {
            return new Hit.Cell(cell);
        }
        Direction side = dx <= dy
            ? (inX < cellSize - inX ? Direction.LEFT : Direction.RIGHT)
            : (inY < cellSize - inY ? Direction.UP : Direction.DOWN);
        return normalized(maze, cell, side);
    }

    /**
     * Применить инструмент к точке холста. Стены ставятся и убираются кликом по границе,
     * предметы, старт и сыр – кликом в клетку (граница клетки тоже считается клеткой).
     */
    public Result apply(Tool tool, double px, double py, double cellSize) {
        Maze maze = controller.maze();
        Result result = switch (tool) {
            case WALL -> wall(maze, hitTest(maze, px, py, cellSize));
            case ERASE -> erase(maze, hitTest(maze, px, py, cellSize));
            case WATER -> item(maze, cellAt(maze, px, py, cellSize), CellType.WATER);
            case SHOCK -> item(maze, cellAt(maze, px, py, cellSize), CellType.SHOCK);
            case START -> start(maze, cellAt(maze, px, py, cellSize));
            case CHEESE -> cheese(maze, cellAt(maze, px, py, cellSize));
        };
        if (result instanceof Result.Changed) {
            controller.mazeEdited();
        }
        return result;
    }

    private static Result wall(Maze maze, Hit hit) {
        if (!(hit instanceof Hit.Edge(Position cell, Direction side))) {
            return new Result.Unchanged();
        }
        if (!maze.contains(cell.step(side))) {
            return new Result.Rejected("Граница лабиринта – всегда стена, её не убрать");
        }
        maze.setWall(cell, side, !maze.hasWall(cell, side));
        return new Result.Changed();
    }

    private static Result erase(Maze maze, Hit hit) {
        return switch (hit) {
            case Hit.Edge(Position cell, Direction side) -> {
                if (!maze.contains(cell.step(side)) || !maze.hasWall(cell, side)) {
                    yield new Result.Unchanged();
                }
                maze.setWall(cell, side, false);
                yield new Result.Changed();
            }
            case Hit.Cell(Position cell) -> {
                CellType type = maze.cellAt(cell);
                if (type != CellType.WATER && type != CellType.SHOCK) {
                    yield new Result.Unchanged();
                }
                maze.setItem(cell, CellType.EMPTY);
                yield new Result.Changed();
            }
            case Hit.Outside() -> new Result.Unchanged();
        };
    }

    /**
     * Вода или ток: в пустую клетку или клетку с другим предметом – положить, с тем же –
     * убрать.
     */
    private static Result item(Maze maze, Position cell, CellType type) {
        if (cell == null) {
            return new Result.Unchanged();
        }
        if (cell.equals(maze.start()) || cell.equals(maze.cheese())) {
            return new Result.Rejected("На старт и на сыр ничего класть нельзя");
        }
    }

```

```

        maze.setItem(cell, maze.cellAt(cell) == type ? CellType.EMPTY : type);
        return new Result.Changed();
    }

    private static Result start(Maze maze, Position cell) {
        if (cell == null || cell.equals(maze.start())) {
            return new Result.Unchanged();
        }
        if (cell.equals(maze.cheese())) {
            return new Result.Rejected("Старт не может стоять на сыре");
        }
        maze.setStart(cell);
        return new Result.Changed();
    }

    private static Result cheese(Maze maze, Position cell) {
        if (cell == null || cell.equals(maze.cheese())) {
            return new Result.Unchanged();
        }
        if (cell.equals(maze.start())) {
            return new Result.Rejected("Сыр не может лежать на старте");
        }
        maze.setCheese(cell);
        return new Result.Changed();
    }

    /** Клетка под точкой или null, если точка вне лабиринта (или числа бессмысленные). */
    private static Position cellAt(Maze maze, double px, double py, double cellSize) {
        if (!(cellSize > 0) || !(px >= 0) || !(py >= 0)) { // так отсекается и NaN
            return null;
        }
        double x = Math.floor(px / cellSize);
        double y = Math.floor(py / cellSize);
        if (x >= maze.width() || y >= maze.height()) {
            return null;
        }
        return new Position((int) x, (int) y);
    }

    /** Внутренняя стена — как правая или нижняя у левой/верхней из двух клеток. */
    private static Hit.Edge normalized(Maze maze, Position cell, Direction side) {
        Position other = cell.step(side);
        if (maze.contains(other) && (side == Direction.LEFT || side == Direction.UP)) {
            return new Hit.Edge(other, side.opposite());
        }
        return new Hit.Edge(cell, side);
    }
}

```

Файл src/main/java/maze/controller/Viewport.java

```

package maze.controller;

/**
 * Масштаб и сдвиг лабиринта на холсте — чистая математика без JavaFX.
 * При масштабе 1 лабиринт вписан в холст и стоит по центру; при увеличении его можно
 * двигать,
 * но край лабиринта не уходит внутрь холста (за лабиринтом нет ничего интересного).
 */
public final class Viewport {

    public static final double MAX_ZOOM = 16;

    /** Где рисовать: левый верхний угол лабиринта и размер клетки, в пикселях холста. */
    public record Layout(double left, double top, double cell) {}

    private final double margin;
    private double zoom = 1;

```

```

/** Сдвиг от положения «по центру», в пикселях. */
private double panX;
private double panY;

/** margin — отступ вокруг лабиринта, чтобы толстая внешняя стена не обрезалась. */
public Viewport(double margin) {
    this.margin = margin;
}

public double zoom() {
    return zoom;
}

/** Раскладка для холста w x h и лабиринта cols x rows; заодно поправляет сдвиг, если
он вышел за края. */
public Layout layout(double w, double h, int cols, int rows) {
    double base = Math.min((w - 2 * margin) / cols, (h - 2 * margin) / rows);
    if (!(base > 0)) {
        return new Layout(0, 0, 0);
    }
    double cell = base * zoom;
    panX = clampPan(panX, w, cols * cell);
    panY = clampPan(panY, h, rows * cell);
    return new Layout((w - cols * cell) / 2 + panX, (h - rows * cell) / 2 + panY,
cell);
}

/** Умножить масштаб на factor так, чтобы точка (px, py) холста осталась над той же
точкой лабиринта. */
public void zoomAt(double factor, double px, double py, double w, double h, int cols,
int rows) {
    if (!(factor > 0)) {
        return; // жест иногда присылает 0 или NaN; бесконечность прижмёт clamp
    }
    Layout before = layout(w, h, cols, rows);
    if (!(before.cell() > 0)) {
        return;
    }
    double mazeX = (px - before.left()) / before.cell();
    double mazeY = (py - before.top()) / before.cell();
    zoom = Math.clamp(zoom * factor, 1, MAX_ZOOM);
    double cell = baseCell(w, h, cols, rows) * zoom;
    panX = px - mazeX * cell - (w - cols * cell) / 2;
    panY = py - mazeY * cell - (h - rows * cell) / 2;
    layout(w, h, cols, rows); // прижать сдвиг к краям
}

/** Сдвинуть лабиринт на (dx, dy) пикселей; не число сбрасывает сдвиг к центру. */
public void pan(double dx, double dy, double w, double h, int cols, int rows) {
    panX += dx;
    panY += dy;
    layout(w, h, cols, rows); // прижать к краям сразу, иначе упор в край копит
«мёртвую зону»
}

/** Снова вписать лабиринт целиком. */
public void reset() {
    zoom = 1; // сдвиг обнулит layout: при масштабе 1 лабиринт помещается и стоит по
центру
}

private double baseCell(double w, double h, int cols, int rows) {
    return Math.min((w - 2 * margin) / cols, (h - 2 * margin) / rows);
}

/** Лабиринт помещается — по центру; не помещается — край не отходит от края холста
дальше отступа. */
private double clampPan(double pan, double size, double content) {
    if (!Double.isFinite(pan) || content <= size - 2 * margin) {
        return 0;
    }

```

```

    }
    double limit = (content - size) / 2 + margin;
    return Math.clamp(pan, -limit, limit);
}
}

```

Файл src/main/java/maze/controller/FormField.java

```

package maze.controller;

import maze.util.SafeText;

import java.math.BigDecimal;
import java.util.regex.Pattern;

/**
 * Поле формы: ключ (имя параметра), подпись для человека и вид значения.
 * Умеет прочитать текст из поля ввода и превратить значение обратно в текст.
 */
public record FormField(String key, String label, Kind kind) {

    public enum Kind {
        INTEGER,
        LONG,
        DECIMAL,
        /** Флажок: окно пишет "true" или "false". */
        YES_NO
    }

    /** Только ASCII-цифры, необязательный знак: «5» или «7» числом не считаются. */
    private static final Pattern INTEGER = Pattern.compile("[+-]?[0-9]+");
    /** Дробь через точку или запятую: «0.5», «0,5», «.5». Экспоненты («1e3») и NaN не принимаются. */
    private static final Pattern DECIMAL = Pattern.compile("[+-]?([0-9]+([.],[0-9]+)?|([.],[0-9]+)?|([.],)[0-9]+)");
    private static final int ECHO_LIMIT = 20;

    /** Прочитанное значение или текст ошибки. */
    public sealed interface Read {
        record Value(Object value) implements Read {}

        record Error(String message) implements Read {}
    }

    public Read read(String raw) {
        String text = raw == null ? "" : raw.strip();
        if (text.isEmpty()) {
            return error("не заполнено");
        }
        String echo = "\"" + SafeText.preview(text, ECHO_LIMIT) + "\"";
        return switch (kind) {
            case INTEGER, LONG -> {
                if (!INTEGER.matcher(text).matches()) {
                    yield error("нужно целое число, а указано " + echo);
                }
                try {
                    yield new Read.Value(kind == Kind.LONG ? (Object) Long.parseLong(text) : Integer.parseInt(text));
                } catch (NumberFormatException e) {
                    yield error("слишком большое число " + echo);
                }
            }
            case DECIMAL -> {
                if (!DECIMAL.matcher(text).matches()) {
                    yield error("нужно число, а указано " + echo);
                }
                double value = Double.parseDouble(text.replace(',', '.'));
            }
        };
    }
}

```

```

        yield Double.isFinite(value) ? new Read.Value(value) : error("слишком
        большое число " + echo);
    }
    case YES_NO -> switch (text) {
        case "true" -> new Read.Value(true);
        case "false" -> new Read.Value(false);
        default -> error("нужно да или нет, а указано " + echo);
    };
};
}

/** Значение как текст для поля ввода: дроби без лишних нулей («0.5», «100»). */
public static String text(Object value) {
    if (value instanceof Double d) {
        return BigDecimal.valueOf(d).stripTrailingZeros().toPlainString();
    }
    return String.valueOf(value);
}

private Read error(String what) {
    return new Read.Error(label + ": " + what);
}
}

```

Файл src/main/java/maze/controller/FormResult.java

```

package maze.controller;

import maze.model.InvalidSettingsException.Problem;

import java.util.List;

/** Результат разбора формы: готовое значение или список понятных проблем. */
public sealed interface FormResult<T> {

    record Valid<T>(T value) implements FormResult<T> {
    }

    record Invalid<T>(List<Problem> problems) implements FormResult<T> {
        public Invalid {
            problems = List.copyOf(problems);
        }
    }
}

```

Файл src/main/java/maze/controller/Forms.java

```

package maze.controller;

import maze.model.InvalidSettingsException;
import maze.model.InvalidSettingsException.Problem;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.function.Function;

/** Общая часть форм: сначала читаются все поля, потом проверяются правила самого объекта. */
final class Forms {

    private Forms() {
    }
}

```

```

/**
 * Если хоть одно поле не прочиталось – возвращаются ошибки чтения (по порядку полей).
 * Иначе значения отдаются build, а его {@link InvalidSettingsException} превращается в
 список проблем.
 */
static <T> FormResult<T> parse(List<FormField> fields, Map<String, String> texts,
Function<Object[], T> build) {
    List<Problem> problems = new ArrayList<>();
    Object[] values = new Object[fields.size()];
    for (int i = 0; i < fields.size(); i++) {
        FormField field = fields.get(i);
        switch (field.read(texts.get(field.key()))) {
            case FormField.Read.Value(Object value) -> values[i] = value;
            case FormField.Read.Error(String message) -> problems.add(new
Problem(field.key(), message));
        }
    }
    if (!problems.isEmpty()) {
        return new FormResult.Invalid<>(problems);
    }
    try {
        return new FormResult.Valid<>(build.apply(values));
    } catch (InvalidSettingsException e) {
        return new FormResult.Invalid<>(e.problems());
    }
}

static Map<String, String> texts(List<FormField> fields, Object... values) {
    Map<String, String> texts = new LinkedHashMap<>();
    for (int i = 0; i < fields.size(); i++) {
        texts.put(fields.get(i).key(), FormField.text(values[i]));
    }
    return texts;
}
}

```

Файл src/main/java/maze/controller/SettingsForm.java

```

package maze.controller;

import maze.model.Settings;

import java.util.List;
import java.util.Map;

import static maze.controller.FormField.Kind.*;

/**
 * Форма настроек обучения без JavaFX: тексты из полей ввода превращаются в {@link
Settings}
 * или в список понятных проблем. Окно параметров только показывает поля и результат.
 */
public final class SettingsForm {

    /** Поля в том же порядке, что и параметры в {@link Settings}. */
    public static final List<FormField> FIELDS = List.of(
        new FormField("cheeseReward", "Награда за сыр", DECIMAL),
        new FormField("waterReward", "Награда за воду", DECIMAL),
        new FormField("shockReward", "Награда за удар током", DECIMAL),
        new FormField("stepReward", "Награда за шаг", DECIMAL),
        new FormField("wallReward", "Награда за удар о стену", DECIMAL),
        new FormField("alpha", "Скорость обучения  $\alpha$ ", DECIMAL),
        new FormField("gamma", "Дисконтирование  $\gamma$ ", DECIMAL),
        new FormField("epsilonStart", "Доля случайных шагов в начале", DECIMAL),
        new FormField("epsilonEnd", "Доля случайных шагов в конце", DECIMAL),
        new FormField("epsilonDecay", "Доля случайных шагов убывает", YES_NO),
        new FormField("decayEpisodes", "За сколько попыток убывает", INTEGER),
        new FormField("maxSteps", "Лимит шагов в попытке", INTEGER),
        new FormField("stableEpisodes", "Автостоп: попыток без изменений пути",
INTEGER),

```

```

        new FormField("seed", "Зерно случайности (seed)", LONG));

private SettingsForm() {
}

/** Значения настроек как тексты для полей формы, в порядке {@link #FIELDS}. */
public static Map<String, String> toTexts(Settings s) {
    return Forms.texts(FIELDS,
        s.cheeseReward(), s.waterReward(), s.shockReward(), s.stepReward(),
s.wallReward(),
        s.alpha(), s.gamma(), s.epsilonStart(), s.epsilonEnd(), s.epsilonDecay(),
        s.decayEpisodes(), s.maxSteps(), s.stableEpisodes(), s.seed());
}

public static FormResult<Settings> parse(Map<String, String> texts) {
    return Forms.parse(FIELDS, texts, v -> Settings.builder()
        .cheeseReward((double) v[0]).waterReward((double)
v[1]).shockReward((double) v[2])
        .stepReward((double) v[3]).wallReward((double) v[4])
        .alpha((double) v[5]).gamma((double) v[6])
        .epsilonStart((double) v[7]).epsilonEnd((double)
v[8]).epsilonDecay((boolean) v[9])
        .decayEpisodes((int) v[10]).maxSteps((int) v[11]).stableEpisodes((int)
v[12])
        .seed((long) v[13])
        .build());
}
}

```

Файл src/main/java/maze/controller/MazeForm.java

```

package maze.controller;

import maze.model.MazeSpec;

import java.util.List;
import java.util.Map;

import static maze.controller.FormField.Kind.*;

/** Форма «Новый лабиринт» без JavaFX: тексты полей → {@link MazeSpec} или список проблем.
 */
public final class MazeForm {

    /** Поля в том же порядке, что и части {@link MazeSpec}. */
    public static final List<FormField> FIELDS = List.of(
        new FormField("width", "Ширина", INTEGER),
        new FormField("height", "Высота", INTEGER),
        new FormField("water", "Клеток с водой", INTEGER),
        new FormField("shock", "Клеток с током", INTEGER),
        new FormField("loopPercent", "Лишних проходов, %", INTEGER),
        new FormField("seed", "Зерно случайности (seed)", LONG));

    private MazeForm() {
    }

    public static Map<String, String> toTexts(MazeSpec s) {
        return Forms.texts(FIELDS, s.width(), s.height(), s.water(), s.shock(),
s.loopPercent(), s.seed());
    }

    public static FormResult<MazeSpec> parse(Map<String, String> texts) {
        return Forms.parse(FIELDS, texts,
            v -> new MazeSpec((int) v[0], (int) v[1], (int) v[2], (int) v[3], (int)
v[4], (long) v[5]));
    }
}

```

Файл src/main/java/maze/controller/SpeedScale.java

```
package maze.controller;

/**
 * Перевод положения слайдера скорости в шаги в секунду и обратно.
 * Шкала логарифмическая: на линейной от 1 до 5000 медленные скорости заняли бы крошечный
 * кусочек слайдера.
 */
public final class SpeedScale {

    public static final double SLIDER_MIN = 0;
    public static final double SLIDER_MAX = 100;

    private static final double LOG_MIN = Math.log(TrainingController.MIN_SPEED);
    private static final double LOG_MAX = Math.log(TrainingController.MAX_SPEED);

    private SpeedScale() {

    }

    /** Положение слайдера → скорость. */
    public static int toSpeed(double slider) {
        double t = (slider - SLIDER_MIN) / (SLIDER_MAX - SLIDER_MIN);
        long speed = Math.round(Math.exp(LOG_MIN + t * (LOG_MAX - LOG_MIN)));
        // За краями слайдера exp даёт почти 0 или бесконечность, NaN округляется в 0 — всё
        // прижимается к диапазону
        return (int) Math.clamp(speed, TrainingController.MIN_SPEED,
            TrainingController.MAX_SPEED);
    }

    /** Скорость → положение слайдера. */
    public static double toSlider(int speed) {
        int clamped = Math.clamp(speed, TrainingController.MIN_SPEED,
            TrainingController.MAX_SPEED);
        return SLIDER_MIN + (Math.log(clamped) - LOG_MIN) / (LOG_MAX - LOG_MIN) *
            (SLIDER_MAX - SLIDER_MIN);
    }
}
```

Файл src/main/java/maze/controller/Texts.java

```
package maze.controller;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** Числа в тексте для человека. */
public final class Texts {

    private Texts() {

    }

    /** «N попытку / попытки / попыток» — для фразы «за N ...». */
    public static String attempts(int n) {
        return n + " " + plural(n, "попытку", "попытки", "попыток");
    }

    /** Форма слова для числа n: одна (1, 21), две-четыре (2-4, 22-24), остальные. */
    private static String plural(int n, String one, String few, String many) {
        int lastTwo = Math.abs(n) % 100;
        int last = lastTwo % 10;
        if (lastTwo >= 11 && lastTwo <= 14) {
            return many;
        }
        if (last == 1) {
            return one;
        }
    }
}
```

```

    }
    return last >= 2 && last <= 4 ? few : many;
}

/** «N шар / шара / шаров». */
public static String steps(int n) {
    return n + " " + plural(n, "шаг", "шара", "шагов");
}

/** Очки со знаком и не больше чем двумя знаками после точки: «+87», «-0.5», «0». Не
число — «-». */
public static String score(double v) {
    if (!Double.isFinite(v)) {
        return "-";
    }
    String abs = plain(Math.abs(v));
    if (abs.equals("0")) {
        return "0";
    }
    return (v > 0 ? "+" : "-") + abs;
}

/** Число без лишних нулей и хвостов вроде 0.30000000000000004, до двух знаков после
точки. */
static String plain(double v) {
    return BigDecimal.valueOf(v).setScale(2,
RoundingMode.HALF_UP).stripTrailingZeros().toPlainString();
}
}

```

Файл src/main/java/maze/util/SafeText.java

```

package maze.util;

/** Безопасный показ введённого пользователем текста внутри сообщений. */
public final class SafeText {

    private SafeText() {
    }

    /**
     * Текст для сообщения об ошибке: не длиннее limit символов (далее «...»),
     * управляющие символы заменены на «?», чтобы они не испортили экран.
     */
    public static String preview(String text, int limit) {
        StringBuilder sb = new StringBuilder();
        text.codePoints()
            .limit(limit)
            .forEach(cp -> sb.appendCodePoint(Character.isISOControl(cp) ? '?' : cp));
        if (text.codePointCount(0, text.length()) > limit) {
            sb.append("...");
        }
        return sb.toString();
    }
}

```

Файл src/main/java/maze/view/MazeApp.java

```

package maze.view;

import javafx.animation.AnimationTimer;
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.CheckBox;
import javafx.scene.control.Label;
import javafx.scene.control.Separator;

```

```

import javafx.scene.control.Slider;
import javafx.scene.control.TextField;
import javafx.scene.control.ToggleButton;
import javafx.scene.control.ToggleGroup;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Priority;
import javafx.scene.layout.Region;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.stage.Stage;
import maze.controller.FormField;
import maze.controller.MazeEditor;
import maze.controller.MazeForm;
import maze.controller.SettingsForm;
import maze.controller.SpeedScale;
import maze.controller.Texts;
import maze.controller.TrainingController;
import maze.model.Attempt;
import maze.model.GreedyPath;
import maze.model.MazeGenerator;
import maze.model.MazeSpec;
import maze.model.Settings;
import maze.model.Trainer;

import java.util.List;
import java.util.Map;

/**
 * Главное окно: лабиринт в центре, управление и график справа, редактор над лабиринтом.
 * Цикл обучения — таймер JavaFX (AnimationTimer): на каждом кадре контроллер решает,
 * сколько шагов сделать, а окно перерисовывает то, что изменилось.
 */
public class MazeApp extends Application {

    /** Лабиринт при запуске программы. */
    static final MazeSpec START_SPEC = MazeSpec.demo();
    /** График перерисовываем не чаще 4 раз в секунду — он дорожке лабиринта. */
    private static final long CHART_INTERVAL_NANOS = 250_000_000L;
    private static final FormField EPISODES = new FormField("episodes", "Число попыток",
FormField.Kind.INTEGER);

    private TrainingController controller;
    private MazeEditor editor;
    private AnimationTimer timer;
    private MazeSpec spec = START_SPEC;
    private boolean edited;

    private final MazeCanvas canvas = new MazeCanvas();
    private final ScoreChart chart = new ScoreChart();
    private final Label mazeInfo = new Label();
    private final Button startPause = new Button();
    private final Button step = new Button("Шаг");
    private final TextField episodes = new TextField("500");
    private final Button train = new Button("Обучить");
    private final Button reset = new Button("Сбросить обучение");
    private final Button replay = new Button("Пройти выученный путь");
    private final Label zoomLabel = new Label();
    private final Slider speed = new Slider(SpeedScale.SLIDER_MIN, SpeedScale.SLIDER_MAX,
0);
    private final Label speedLabel = new Label();
    private final CheckBox showPath = new CheckBox("выученный путь");
    private final CheckBox showArrows = new CheckBox("стрелки направлений");
    private final CheckBox autoStop = new CheckBox("стоп, когда выучила");
    private final Label attemptValue = new Label();
    private final Label scoreValue = new Label();
    private final Label stepsValue = new Label();
    private final Label randomValue = new Label();
    private final Label lastValue = new Label();
    private final Label status = new Label();

```

```

private final Label message = new Label();
private final ToggleButton editorToggle = new ToggleButton("Редактор");
private final ToggleGroup tools = new ToggleGroup();
private final Label legendWater = new Label();
private final Label legendShock = new Label();
private final Label legendCheese = new Label();
private HBox editorBar;

private long lastDrawnSteps = Long.MIN_VALUE;
private long lastChartUpdate;

static String stylesheet() {
    return MazeApp.class.getResource("app.css").toExternalForm();
}

@Override
public void start(Stage stage) {
    controller = new TrainingController(MazeGenerator.generate(spec),
Settings.defaults());
    editor = new MazeEditor(controller);

    editorBar = buildEditorBar();
    VBox center = new VBox(8, editorBar, canvas, buildLegend());
    VBox.setVgrow(canvas, Priority.ALWAYS);
    center.setPadding(new Insets(10));

    BorderPane root = new BorderPane();
    root.setTop(buildToolbar());
    root.setCenter(center);
    root.setRight(buildPanel());

    Scene scene = new Scene(root, 1180, 780);
    scene.getStylesheets().add(stylesheet());
    stage.setTitle("Мышь в лабиринте");
    stage.setMinWidth(860);
    stage.setMinHeight(600);
    stage.setScene(scene);

    canvas.setOnMouseClicked(e -> {
        // клик — только если мышь не сдвигалась: перетаскивание двигает лабиринт, а не
        ставит стены
        if (e.isStillSincePress() && editorToggle.isSelected() &&
tools.getSelectedToggle() != null) {
            MazeEditor.Tool tool = (MazeEditor.Tool)
tools.getSelectedToggle().getUserData();
            MazeEditor.Result result = editor.apply(tool,
                canvas.toMazeX(e.getX()), canvas.toMazeY(e.getY()),
canvas.cellSize());
            switch (result) {
                case MazeEditor.Result.Changed() -> {
                    edited = true;
                    message.setText("");
                }
                case MazeEditor.Result.Rejected(String why) -> message.setText(why);
                case MazeEditor.Result.Unchanged() -> {
                }
            }
            refresh(true, System.nanoTime());
        }
    });

    timer = new AnimationTimer() {
        @Override
        public void handle(long now) {
            controller.tick(now);
            refresh(false, now);
        }
    };
    refresh(true, System.nanoTime());
    stage.show();
}

```

```

        timer.start();
    }

    @Override
    public void stop() {
        if (timer != null) {
            timer.stop();
        }
    }

    private HBox buildToolbar() {
        Label title = new Label("Мышь в лабиринте");
        title.getStyleClass().add("title");
        mazeInfo.getStyleClass().add("muted");
        Region spacer = new Region();
        HBox.setHgrow(spacer, Priority.ALWAYS);

        Button newMaze = new Button("Новый лабиринт...");
        newMaze.setId("newMaze");
        newMaze.setOnAction(e -> openNewMaze());
        editorToggle.setId("editor");
        editorToggle.setOnAction(e -> {
            controller.pause();
            controller.stopReplay();
            canvas.setDragPan(!editorToggle.isSelected());
            message.setText("");
            refresh(true, System.nanoTime());
        });
        Button parameters = new Button("Параметры...");
        parameters.setId("parameters");
        parameters.setOnAction(e -> openParameters());

        HBox bar = new HBox(10, title, mazeInfo, spacer, newMaze, editorToggle,
parameters);
        bar.setAlignment(Pos.CENTER_LEFT);
        bar.getStyleClass().add("toolbar-row");
        return bar;
    }

    private HBox buildEditorBar() {
        HBox bar = new HBox(6);
        bar.setAlignment(Pos.CENTER_LEFT);
        bar.getStyleClass().add("editor-bar");
        String[][] names = {{ "WALL", "Стена"}, {"WATER", "Вода"}, {"SHOCK", "Ток"},
            {"START", "Старт"}, {"CHEESE", "Сыр"}, {"ERASE", "Ластик"} };
        for (String[] n : names) {
            ToggleButton b = new ToggleButton(n[1]);
            b.setId("tool-" + n[0]);
            b.setUserData(MazeEditor.Tool.valueOf(n[0]));
            b.setToggleGroup(tools);
            b.setMinWidth(Region.USE_PREF_SIZE);
            bar.getChildren().add(b);
        }
        tools.getToggles().getFirst().setSelected(true);
        // инструмент нельзя «отжать» совсем: повторный клик по выбранному оставляет его
выбранным
        tools.selectedToggleProperty().addListener((obs, old, now) -> {
            if (now == null) {
                old.setSelected(true);
            }
        });
        Label hint = new Label("Стена — клик по границе клеток; остальное — клик в клетку.
Изменения сбрасывают обучение.");
        hint.getStyleClass().add("muted");
        hint.setWrapText(true);
        hint.setMinWidth(0);
        HBox.setHgrow(hint, Priority.ALWAYS);
        bar.getChildren().add(hint);
        bar.visibleProperty().bind(editorToggle.selectedProperty());
        bar.managedProperty().bind(editorToggle.selectedProperty());
        return bar;
    }
}

```

```

private HBox buildLegend() {
    HBox legend = new HBox(16,
        legendItem(MazeCanvas.WATER_TILE, MazeCanvas.WATER, legendWater),
        legendItem(MazeCanvas.SHOCK_TILE, MazeCanvas.SHOCK, legendShock),
        legendItem(MazeCanvas.CHEESE_TILE, MazeCanvas.CHEESE, legendCheese),
        legendItem(MazeCanvas.PATH, MazeCanvas.PATH, new Label("выученный путь")));
    Region spacer = new Region();
    HBox.setHgrow(spacer, Priority.ALWAYS);
    Button fit = new Button("Сбросить масштаб");
    fit.setId("fit");
    fit.setOnAction(e -> canvas.resetView());
    canvas.setOnViewChanged(this::updateZoomLabel);
    updateZoomLabel();
    legend.getChildren().addAll(spacer, zoomLabel, fit);
    legend.setAlignment(Pos.CENTER_LEFT);
    legend.getStyleClass().add("legend");
    return legend;
}

private void updateZoomLabel() {
    zoomLabel.setText("Масштаб: " + Math.round(canvas.zoom() * 100) + "%");
}

private static HBox legendItem(Color fill, Color stroke, Label label) {
    Rectangle square = new Rectangle(12, 12, fill);
    square.setStroke(stroke);
    HBox row = new HBox(6, square, label);
    row.setAlignment(Pos.CENTER_LEFT);
    return row;
}

private VBox buildPanel() {
    startPause.setId("startPause");
    step.setId("step");
    train.setId("train");
    episodes.setId("episodes");
    reset.setId("reset");
    replay.setId("replay");
    speed.setId("speed");
    showPath.setId("showPath");
    showArrows.setId("showArrows");
    autoStop.setId("autoStop");

    startPause.setMaxWidth(Double.MAX_VALUE);
    step.setMaxWidth(Double.MAX_VALUE);
    HBox.setHgrow(startPause, Priority.ALWAYS);
    HBox.setHgrow(step, Priority.ALWAYS);
    HBox run = new HBox(6, startPause, step);

    episodes.setPrefColumnCount(6);
    train.setMaxWidth(Double.MAX_VALUE);
    HBox.setHgrow(train, Priority.ALWAYS);
    HBox trainRow = new HBox(6, episodes, train);
    reset.setMaxWidth(Double.MAX_VALUE);
    replay.setMaxWidth(Double.MAX_VALUE);

    startPause.setOnAction(e -> command(controller::toggleRunning));
    step.setOnAction(e -> command(controller::stepOnce));
    train.setOnAction(e -> trainEpisodes());
    episodes.setOnAction(e -> trainEpisodes());
    reset.setOnAction(e -> command(controller::reset));
    replay.setOnAction(e -> {
        message.setText("");
        if (!controller.startReplay()) {
            message.setText("Мышь ещё не знает пути до сыра — сначала обучите её");
        }
        refresh(true, System.nanoTime());
    });
}

```

```

speed.setValue(SpeedScale.toSlider(controller.getSpeed()));
speed.valueProperty().addListener((obs, old, now) -> {
    controller.setSpeed(SpeedScale.toSpeed(now.doubleValue()));
    updateSpeedLabel();
});
updateSpeedLabel();

showPath.setSelected(true);
autoStop.setSelected(controller.isAutoStop());
autoStop.setOnAction(e -> controller.setAutoStop(autoStop.isSelected()));
showPath.setOnAction(e -> refresh(true, System.nanoTime()));
showArrows.setOnAction(e -> refresh(true, System.nanoTime()));

GridPane stats = new GridPane();
stats.setHgap(6);
stats.setVgap(6);
stats.add(stat("Попытка", attemptValue), 0, 0);
stats.add(stat("Очки", scoreValue), 1, 0);
stats.add(stat("Шаров", stepsValue), 0, 1);
stats.add(stat("Наград", randomValue), 1, 1);
lastValue.getStyleClass().add("muted");

status.getStyleClass().add("status");
status.setWrapText(true);
status.setMinHeight(Region.USE_PREF_SIZE);    // две строки не обрезаются графиком
снизу
message.getStyleClass().add("message");
message.setWrapText(true);
message.setMinHeight(Region.USE_PREF_SIZE);

Label chartTitle = new Label("Сумма выигрыша за попытку");
chartTitle.getStyleClass().add("muted");
chart.setMinHeight(180);
VBox.setVgrow(chart, Priority.ALWAYS);

VBox box = new VBox(8,
    run, trainRow, reset, replay,
    new Separator(), speedLabel, speed, showPath, showArrows, autoStop,
    new Separator(), stats, lastValue, status, message,
    new Separator(), chartTitle, chart);
box.getStyleClass().add("panel");
// ширина панели постоянная: длинное сообщение переносится, а не раздвигает панель
и график
box.setPrefWidth(290);
box.setMinWidth(290);
box.setMaxWidth(290);
return box;
}

private static VBox stat(String name, Label value) {
    Label label = new Label(name);
    label.getStyleClass().add("stat-name");
    value.getStyleClass().add("stat-value");
    VBox card = new VBox(0, label, value);
    card.getStyleClass().add("stat");
    card.setPrefWidth(130);
    return card;
}

/** Нажатие кнопки: выполнить действие и сразу показать результат. */
private void command(Runnable action) {
    message.setText("");
    action.run();
    refresh(true, System.nanoTime());
}

private void trainEpisodes() {
    switch (EPISODES.read(episodes.getText())) {
        case FormField.Read.Error(String why) -> message.setText(why);
        case FormField.Read.Value(Object value) -> {

```

```

        int n = (int) value;
        if (n < 1 || n > TrainingController.MAX_EPISODES_AT_ONCE) {
            message.setText("Число попыток: от 1 до " +
TrainingController.MAX_EPISODES_AT_ONCE);
            break;
        }
        int done = controller.trainEpisodes(n);
        message.setText(done == n
            ? "Мышь пробежала ещё " + Texts.attempts(done)
            : "Мышь пробежала " + Texts.attempts(done) + " из " + n + ":
остальное заняло бы слишком долго");
    }
    refresh(true, System.nanoTime());
}

private void openParameters() {
    controller.pause();
    Map<String, String> sections = Map.of(
        "cheeseReward", "Награды (среда сообщает мыши)",
        "alpha", "Обучение (Q-learning)",
        "maxSteps", "Попытки");
    new FormDialog<>("Параметры обучения", "После применения мышь учится заново.",
"Применить",
        SettingsForm.FIELDS, sections,
        SettingsForm.toTexts(controller.settings()),
SettingsForm.toTexts(Settings.defaults()),
        SettingsForm::parse, Settings::warnings)
        .showAndWait().ifPresent(controller::applySettings);
    message.setText("");
    refresh(true, System.nanoTime());
}

private void openNewMaze() {
    controller.pause();
    new FormDialog<>("Новый лабиринт", "Лабиринт строится случайно по seed; потом его
можно поправить в редакторе.",
        "Построить", MazeForm.FIELDS, Map.of(),
        MazeForm.toTexts(spec), MazeForm.toTexts(START_SPEC), MazeForm::parse,
newSpec -> List.of())
        .showAndWait().ifPresent(newSpec -> {
            spec = newSpec;
            edited = false;
            controller.setMaze(MazeGenerator.generate(newSpec));
        });
    message.setText("");
    refresh(true, System.nanoTime());
}

private void updateSpeedLabel() {
    speedLabel.setText("Скорость: " + controller.getSpeed() + " шар./с");
}

private void refresh(boolean force, long now) {
    Trainer trainer = controller.trainer();
    // во время показа выученного пути на экране — попытка показа, а не обучения
    Attempt shown = controller.replay() != null ? controller.replay() :
trainer.attempt();
    long drawKey = trainer.totalSteps() * 100_003 + (controller.replay() == null ? -1 :
controller.replay().steps());
    boolean changed = drawKey != lastDrawnSteps;
    if (force || changed) {
        // обрывок пути (мышь ещё не знает дороги) выглядел бы как выученный путь — его
не рисуем
        GreedyPath path = shown.isSelected() ? trainer.bestPath() : null;
        canvas.draw(controller.maze(), shown, trainer.table(),
            path != null && path.reachesCheese() ? path : null,
shown.isSelected());
        boolean replaying = controller.replay() != null;
        attemptValue.setText(replaying ? "показ" :
Integer.toString(trainer.episode()));
        scoreValue.setText(Texts.score(shown.score()));
    }
}

```

```

        stepsValue.setText(Integer.toString(shown.steps()));
        randomValue.setText(replaying ? "0%" : Math.round(trainer.epsilon() * 100) +
"%");

        lastValue.setText("Прошлая попытка: " + Texts.score(trainer.lastScore()));
        status.setText(controller.statusMessage());
        lastDrawnSteps = drawKey;
    }
    if (force || (changed && now - lastChartUpdate >= CHART_INTERVAL_NANOS)) {
        chart.update(trainer.history());
        lastChartUpdate = now;
    }
    Settings s = controller.settings();
    legendWater.setText("вода " + Texts.score(s.waterReward()));
    legendShock.setText("ток " + Texts.score(s.shockReward()));
    legendCheese.setText("сыр " + Texts.score(s.cheeseReward()));
    mazeInfo.setText(controller.maze().width() + " × " + controller.maze().height()
        + " · seed " + spec.seed() + (edited ? " · изменён" : ""));
    startPause.setText(controller.isRunning() ? "Пауза" : "Старт");
}
}

```

Файл src/main/java/maze/view/MazeCanvas.java

```

package maze.view;

import javafx.scene.Cursor;
import javafx.scene.canvas.Canvas;
import javafx.scene.canvas.GraphicsContext;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.StrokeLineCap;
import javafx.scene.shape.StrokeLineJoin;
import maze.controller.Viewport;
import maze.model.Attempt;
import maze.model.CellType;
import maze.model.Direction;
import maze.model.GreedyPath;
import maze.model.Maze;
import maze.model.Position;
import maze.model.QTable;

import java.util.List;

/**
 * Лабиринт на экране: клетки с водой, током и сыром, тонкие стены, мышь,
 * по желанию — выученный путь и стрелки лучших направлений.
 * Растягивается под размер окна, клетки остаются квадратными.
 * <p>
 * Масштаб и сдвиг — как в картах на Мас: щипок двумя пальцами приближает, движение двумя
 * пальцами
 * (или колесо) двигает, Cmd + колесо приближает, перетаскивание мышью двигает (кроме
 * режима редактора).
 */
public class MazeCanvas extends Pane {

    static final Color BACKGROUND = Color.web("#ffffff");
    static final Color FLOOR = Color.web("#f1efe8");
    static final Color WALL = Color.web("#444441");
    static final Color WATER_TILE = Color.web("#e6f1fb");
    static final Color WATER = Color.web("#185fa5");
    static final Color SHOCK_TILE = Color.web("#fcebeb");
    static final Color SHOCK = Color.web("#a32d2d");
    static final Color CHEESE_TILE = Color.web("#faeeda");
    static final Color CHEESE = Color.web("#ba7517");
    static final Color PATH = Color.web("#1d9e75");
    static final Color MOUSE = Color.web("#185fa5");
    static final Color ARROW = Color.web("#b4b2a9");
    static final Color START = Color.web("#888780");

    /** Отступ вокруг лабиринта, чтобы толстая внешняя стена не обрезалась. */
    private static final double MARGIN = 6;

```

```

private final Canvas canvas = new Canvas();
private final Viewport viewport = new Viewport(MARGIN);
/** Можно ли двигать лабиринт перетаскиванием (в редакторе мышшь ставит стены). */
private boolean dragPan = true;
private double dragX;
private double dragY;
private Runnable onViewChanged = () -> { };
private Maze maze;
private Attempt attempt;
private QTable table;
private GreedyPath path;
private boolean arrows;

/** Где сейчас нарисован лабиринт: левый верхний угол и размер клетки. */
private double left;
private double top;
private double cell;

public MazeCanvas() {
    getChildren().add(canvas);
    getStyleClass().add("maze-canvas");

    setOnZoom(e -> {
        zoomAt(e.getZoomFactor(), e.getX(), e.getY());
        e.consume();
    });
    setOnScroll(e -> {
        if (e.isShortcutDown() || e.isControlDown()) {
            zoomAt(Math.exp(e.getDeltaY() * 0.01), e.getX(), e.getY()); // Cmd +
колесо
        } else {
            panBy(e.getDeltaX(), e.getDeltaY()); // два
пальца / колесо
        }
        e.consume();
    });
    setOnMousePressed(e -> {
        dragX = e.getX();
        dragY = e.getY();
    });
    setOnMouseDragged(e -> {
        if (dragPan) {
            setCursor(Cursor.CLOSED_HAND);
            panBy(e.getX() - dragX, e.getY() - dragY);
            dragX = e.getX();
            dragY = e.getY();
        }
    });
    setOnMouseReleased(e -> setCursor(Cursor.DEFAULT));
}

/** Двигать ли лабиринт перетаскиванием мыши (в редакторе — нет, там мышшь рисует). */
public void setDragPan(boolean dragPan) {
    this.dragPan = dragPan;
}

/** Вызывается после изменения масштаба или сдвига (окно обновляет надпись с
масштабом). */
public void setOnViewChanged(Runnable onViewChanged) {
    this.onViewChanged = onViewChanged;
}

/** Масштаб: 1 — лабиринт вписан целиком. */
public double zoom() {
    return viewport.zoom();
}

/** Кнопка «Сбросить масштаб»: снова весь лабиринт целиком. */
public void resetView() {

```

```

        viewport.reset();
        redraw();
        onViewChanged.run();
    }

    private void zoomAt(double factor, double x, double y) {
        if (maze != null) {
            viewport.zoomAt(factor, x, y, canvas.getWidth(), canvas.getHeight(),
                maze.width(), maze.height());
            redraw();
            onViewChanged.run();
        }
    }

    private void panBy(double dx, double dy) {
        if (maze != null) {
            viewport.pan(dx, dy, canvas.getWidth(), canvas.getHeight(), maze.width(),
                maze.height());
            redraw();
        }
    }

    /**
     * Нарисовать состояние (запоминается — при изменении размера окна перерисовывается само).
     *
     * @param path    выученный путь или null, если его не показывать
     * @param arrows  рисовать ли стрелки лучших направлений
     */
    public void draw(Maze maze, Attempt attempt, QTable table, GreedyPath path, boolean
        arrows) {
        if (this.maze == null || this.maze.width() != maze.width() || this.maze.height() !=
            maze.height()) {
            viewport.reset(); // новый размер — снова вписать целиком
            onViewChanged.run();
        }
        this.maze = maze;
        this.attempt = attempt;
        this.table = table;
        this.path = path;
        this.arrows = arrows;
        redraw();
    }

    /** Координата холста → координата внутри лабиринта (для редактора): x. */
    public double toMazeX(double canvasX) {
        return canvasX - left;
    }

    public double toMazeY(double canvasY) {
        return canvasY - top;
    }

    public double cellSize() {
        return cell;
    }

    @Override
    protected void layoutChildren() {
        canvas.setWidth(getWidth());
        canvas.setHeight(getHeight());
        redraw();
    }

    // Размер холста не должен влиять на желаемый размер панели — иначе окно нельзя
    уменьшить.
    @Override
    protected double computePrefWidth(double height) {
        return 720;
    }

```

```

@Override
protected double computePrefHeight(double width) {
    return 540;
}

@Override
protected double computeMinWidth(double height) {
    return 120;
}

@Override
protected double computeMinHeight(double width) {
    return 90;
}

private void redraw() {
    double width = canvas.getWidth();
    double height = canvas.getHeight();
    GraphicsContext g = canvas.getGraphicsContext2D();
    g.setFill(BACKGROUND);
    g.fillRect(0, 0, width, height);
    if (maze == null) {
        return;
    }
    Viewport.Layout layout = viewport.layout(width, height, maze.width(),
    maze.height());
    if (!(layout.cell() > 0)) {
        return;
    }
    cell = layout.cell();
    left = Math.floor(layout.left()); // целые пиксели — стены чёткие, а не размытые
    top = Math.floor(layout.top());

    g.setFill(FLOOR);
    g.fillRect(left, top, cell * maze.width(), cell * maze.height());
    drawCells(g);
    if (arrows && table != null && cell >= 14) {
        drawArrows(g);
    }
    if (path != null) {
        drawPath(g, path.cells());
    }
    drawWalls(g);
    if (attempt != null) {
        drawMouse(g, attempt.position());
    }
}

private void drawCells(GraphicsContext g) {
    double inset = cell >= 8 ? Math.max(1, cell * 0.06) : 0;
    for (int y = 0; y < maze.height(); y++) {
        for (int x = 0; x < maze.width(); x++) {
            Position p = new Position(x, y);
            CellType type = maze.cellAt(p);
            if (type == CellType.EMPTY) {
                continue;
            }
            double px = left + x * cell;
            double py = top + y * cell;
            boolean faded = type == CellType.WATER && attempt != null &&
!attempt.hasWater(p);
            g.setGlobalAlpha(faded ? 0.35 : 1);
            g.setFill(switch (type) {
                case WATER -> WATER_TILE;
                case SHOCK -> SHOCK_TILE;
                default -> CHEESE_TILE;
            });
            g.fillRect(px + inset, py + inset, cell - 2 * inset, cell - 2 * inset);
            if (cell >= 10) {
                switch (type) {

```

```

        case WATER -> Icons.drop(g, px, py, cell, WATER);
        case SHOCK -> Icons.bolt(g, px, py, cell, SHOCK);
        default -> Icons.cheese(g, px, py, cell, CHEESE);
    }
} else {
    // мелкие клетки: значок не разглядеть, остаётся насыщенный цвет
    g.setFill(switch (type) {
        case WATER -> WATER;
        case SHOCK -> SHOCK;
        default -> CHEESE;
    });
    g.fillRect(px, py, cell, cell);
}
g.setGlobalAlpha(1);
}
}
// старт – пунктирное кольцо, чтобы было видно, откуда начинается каждая попытка
Position s = maze.start();
if (cell >= 8) {
    g.setStroke(START);
    g.setLineWidth(Math.max(1, cell * 0.05));
    g.setLineDashes(Math.max(2, cell * 0.12));
    double r = cell * 0.36;
    g.strokeOval(left + s.x() * cell + cell / 2 - r, top + s.y() * cell + cell / 2
- r, 2 * r, 2 * r);
    g.setLineDashes();
}
}

private void drawArrows(GraphicsContext g) {
    g.setStroke(ARROW);
    g.setLineWidth(Math.max(1, cell * 0.06));
    g.setLineCap(StrokeLineCap.ROUND);
    for (int y = 0; y < maze.height(); y++) {
        for (int x = 0; x < maze.width(); x++) {
            Position p = new Position(x, y);
            if (p.equals(maze.cheese()) || table.bestAll(p).size() ==
Direction.values().length) {
                continue; // у сыра шагать некуда; все числа равны – мышь тут ещё
ничего не знает
            }
            Direction d = table.best(p);
            double cx = left + x * cell + cell / 2;
            double cy = top + y * cell + cell / 2;
            double len = cell * 0.28;
            double ex = cx + d.dx() * len;
            double ey = cy + d.dy() * len;
            g.strokeLine(cx - d.dx() * len * 0.6, cy - d.dy() * len * 0.6, ex, ey);
            double head = cell * 0.12;
            // два уса наконечника: повернуть направление назад и вбок
            g.strokeLine(ex, ey, ex - d.dx() * head - d.dy() * head, ey - d.dy() * head
+ d.dx() * head);
            g.strokeLine(ex, ey, ex - d.dx() * head + d.dy() * head, ey - d.dy() * head
- d.dx() * head);
        }
    }
}

private void drawPath(GraphicsContext g, List<Position> cells) {
    if (cells.size() < 2) {
        return;
    }
    g.setStroke(PATH);
    g.setLineWidth(Math.max(2, cell * 0.12));
    g.setLineCap(StrokeLineCap.ROUND);
    g.setLineJoin(StrokeLineJoin.ROUND);
    g.beginPath();
    for (int i = 0; i < cells.size(); i++) {
        Position p = cells.get(i);
        double px = left + p.x() * cell + cell / 2;
        double py = top + p.y() * cell + cell / 2;
        if (i == 0) {
            g.moveTo(px, py);
        } else {

```

```

        g.lineTo(px, py);
    }
}
g.stroke();
}

private void drawWalls(GraphicsContext g) {
    g.setStroke(WALL);
    g.setLineCap(StrokeLineCap.ROUND);
    g.setLineWidth(Math.max(1, Math.min(3, cell * 0.08)));
    g.beginPath();
    for (int y = 0; y < maze.height(); y++) {
        for (int x = 0; x < maze.width(); x++) {
            Position p = new Position(x, y);
            double px = left + x * cell;
            double py = top + y * cell;
            if (x < maze.width() - 1 && maze.hasWall(p, Direction.RIGHT)) {
                g.moveTo(px + cell, py);
                g.lineTo(px + cell, py + cell);
            }
            if (y < maze.height() - 1 && maze.hasWall(p, Direction.DOWN)) {
                g.moveTo(px, py + cell);
                g.lineTo(px + cell, py + cell);
            }
        }
    }
    g.stroke();
    g.setLineWidth(Math.max(2, Math.min(4, cell * 0.12)));
    g.strokeRect(left, top, cell * maze.width(), cell * maze.height());
}

private void drawMouse(GraphicsContext g, Position p) {
    double r = Math.max(2, cell * 0.28);
    double cx = left + p.x() * cell + cell / 2;
    double cy = top + p.y() * cell + cell / 2;
    g.setFill(MOUSE);
    if (cell >= 14) {
        // уши
        double ear = r * 0.45;
        g.fillOval(cx - r * 0.85 - ear, cy - r * 0.85 - ear, 2 * ear, 2 * ear);
        g.fillOval(cx + r * 0.85 - ear, cy - r * 0.85 - ear, 2 * ear, 2 * ear);
    }
    g.fillOval(cx - r, cy - r, 2 * r, 2 * r);
}
}

```

Файл src/main/java/maze/view/Icons.java

```

package maze.view;

import javafx.scene.canvas.GraphicsContext;
import javafx.scene.paint.Color;

/** Значки клеток, нарисованные линиями: капля, молния, кусок сыра. (x, y) — левый верхний
    угол клетки. */
final class Icons {

    private Icons() {
    }

    static void drop(GraphicsContext g, double x, double y, double size, Color color) {
        double cx = x + size / 2;
        double r = size * 0.18;
        double cy = y + size * 0.58;
        g.setFill(color);
        g.beginPath();
        g.moveTo(cx, y + size * 0.2); // острый кончик сверху
        g.bezierCurveTo(cx + r * 0.4, cy - r * 1.2, cx + r, cy - r * 0.6, cx + r, cy);
        g.arc(cx, cy, r, r, 0, -180); // круглое донышко
    }
}

```

```

        g.bezierCurveTo(cx - r, cy - r * 0.6, cx - r * 0.4, cy - r * 1.2, cx, y + size *
0.2);
        g.closePath();
        g.fill();
    }

    static void bolt(GraphicsContext g, double x, double y, double size, Color color) {
        double[] px = {0.56, 0.30, 0.48, 0.40, 0.70, 0.52};
        double[] py = {0.18, 0.56, 0.56, 0.82, 0.44, 0.44};
        double[] xs = new double[px.length];
        double[] ys = new double[py.length];
        for (int i = 0; i < px.length; i++) {
            xs[i] = x + px[i] * size;
            ys[i] = y + py[i] * size;
        }
        g.setFill(color);
        g.fillPolygon(xs, ys, xs.length);
    }

    static void cheese(GraphicsContext g, double x, double y, double size, Color color) {
        // клин сыра и две дырки
        double[] xs = {x + size * 0.2, x + size * 0.8, x + size * 0.8};
        double[] ys = {y + size * 0.7, y + size * 0.3, y + size * 0.7};
        g.setFill(color);
        g.fillPolygon(xs, ys, 3);
        g.setFill(MazeCanvas.CHEESE_TILE);
        double h = size * 0.07;
        g.fillOval(x + size * 0.62 - h, y + size * 0.58 - h, 2 * h, 2 * h);
        g.fillOval(x + size * 0.70 - h * 0.7, y + size * 0.45 - h * 0.7, 1.4 * h, 1.4 * h);
    }
}

```

Файл src/main/java/maze/view/ScoreChart.java

```

package maze.view;

import javafx.scene.chart.LineChart;
import javafx.scene.chart.NumberAxis;
import javafx.scene.chart.XYChart;
import javafx.util.StringConverter;

import java.util.ArrayList;
import java.util.List;

/**
 * График «сумма выигрыша за попытку». Бледная линия — каждая попытка,
 * яркая — среднее за последние {@value #WINDOW} попыток: по ней видно, что мышь учится.
 */
public class ScoreChart extends LineChart<Number, Number> {

    /** Больше точек на линию не рисуем: график остаётся быстрым при любой длине истории.
    */
    private static final int MAX_POINTS = 300;
    static final int WINDOW = 20;

    private final XYChart.Series<Number, Number> raw = new XYChart.Series<>();
    private final XYChart.Series<Number, Number> average = new XYChart.Series<>();

    public ScoreChart() {
        super(new NumberAxis(), new NumberAxis());
        setAnimated(false);
        setCreateSymbols(false);
        setLegendVisible(true);
        getXAxis().setLabel("Попытка");
        NumberAxis attempts = (NumberAxis) getXAxis();
        attempts.setForceZeroInRange(false);
        // Номер попытки — целое число: «10», а не «10.00»
        attempts.setTickLabelFormatter(new StringConverter<>() {
            @Override

```

```

        public String toString(Number value) {
            return Long.toString(Math.round(value.doubleValue()));
        }

        @Override
        public Number fromString(String text) {
            return Long.parseLong(text);
        }
    });
    raw.setName("за попытку");
    average.setName("среднее за " + WINDOW);
    getData().addAll(List.of(raw, average));
}

/** history – суммы законченных попыток по порядку (попытка i+1 – элемент i). */
public void update(List<Double> history) {
    int n = history.size();
    double[] prefix = new double[n + 1];
    for (int i = 0; i < n; i++) {
        prefix[i + 1] = prefix[i] + history.get(i);
    }
    List<XYChart.Data<Number, Number>> r = new ArrayList<>();
    List<XYChart.Data<Number, Number>> a = new ArrayList<>();
    int stride = Math.max(1, (int) Math.ceil(n / (double) MAX_POINTS));
    for (int i = 0; i < n; i++) {
        // каждая stride-я точка, но последняя – всегда
        if (i % stride != 0 && i != n - 1) {
            continue;
        }
        int from = Math.max(0, i + 1 - WINDOW);
        r.add(new XYChart.Data<>(i + 1, history.get(i)));
        a.add(new XYChart.Data<>(i + 1, (prefix[i + 1] - prefix[from]) / (i + 1 -
from)));
    }
    raw.getData().setAll(r);
    average.getData().setAll(a);
}
}

```

Файл src/main/java/maze/view/FormDialog.java

```

package maze.view;

import javafx.event.ActionEvent;
import javafx.geometry.Insets;
import javafx.scene.control.Button;
import javafx.scene.control.ButtonBar;
import javafx.scene.control.ButtonType;
import javafx.scene.control.CheckBox;
import javafx.scene.control.Control;
import javafx.scene.control.Dialog;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.ColumnConstraints;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Region;
import javafx.scene.layout.VBox;
import maze.controller.FormField;
import maze.controller.FormResult;
import maze.model.InvalidSettingsException.Problem;

import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Random;
import java.util.function.Function;

/**
 * Окно с формой. Проверка введённого – в контроллере (SettingsForm, MazeForm);

```

```

* здесь только поля, подсветка ошибок и кнопки. С ошибками окно не закрывается.
*/
class FormDialog<T> extends Dialog<T> {

    private final Map<String, Control> inputs = new LinkedHashMap<>();
    private final Label errors = new Label();
    private final Function<Map<String, String>, FormResult<T>> parser;
    private final Function<T, List<String>> warningsOf;
    private final Label warnings = new Label();
    private Button applyButton;
    private String applyText;
    private T accepted;
    /** Значение, о котором уже предупредили: второе нажатие «Применить» с ним же —
    согласие. */
    private T warned;

    /**
     * @param sections подзаголовки: ключ поля → заголовок группы, которая с него
    начинается
     * @param defaults тексты для кнопки «По умолчанию»
     */
    FormDialog(String title, String header, String applyText,
        List<FormField> fields, Map<String, String> sections,
        Map<String, String> current, Map<String, String> defaults,
        Function<Map<String, String>, FormResult<T>> parser,
        Function<T, List<String>> warningsOf) {
        this.parser = parser;
        this.warningsOf = warningsOf;
        this.applyText = applyText;
        setTitle(title);
        setHeaderText(header);

        GridPane grid = new GridPane();
        grid.setHgap(10);
        grid.setVgap(6);
        grid.setPadding(new Insets(4, 4, 4, 4));
        // Подписи никогда не обрезаются многоточием: столбец не уже самой длинной подписи
        ColumnConstraints labelColumn = new ColumnConstraints();
        labelColumn.setMinWidth(Region.USE_PREF_SIZE);
        grid.getColumnConstraints().addAll(labelColumn, new ColumnConstraints(), new
    ColumnConstraints());
        int row = 0;
        for (FormField field : fields) {
            String section = sections.get(field.key());
            if (section != null) {
                Label heading = new Label(section);
                heading.getStyleClass().add("section");
                grid.add(heading, 0, row++, 3, 1);
            }
            Control input;
            if (field.kind() == FormField.Kind.YES_NO) {
                CheckBox box = new CheckBox();
                box.setSelected(Boolean.parseBoolean(current.get(field.key())));
                input = box;
            } else {
                TextField text = new TextField(current.get(field.key()));
                text.setPrefColumnCount(10);
                input = text;
            }
            input.setId("field-" + field.key());
            inputs.put(field.key(), input);
            grid.add(new Label(field.label()), 0, row);
            grid.add(input, 1, row);
            if (field.key().equals("seed")) {
                Button random = new Button("Случайный");
                random.setMinWidth(Region.USE_PREF_SIZE);
                random.setOnAction(e -> ((TextField) input).setText(Long.toString(new
    Random().nextLong())));
                grid.add(random, 2, row);
            }
            row++;
        }
    }
}

```

```

Button reset = new Button("По умолчанию");
reset.setOnAction(e -> fill(defaults));

errors.getStyleClass().add("error-text");
errors.setWrapText(true);
errors.setMaxWidth(520);
// Блок ошибок растёт по высоте под все строки, а не обрезается
errors.setMinHeight(Region.USE_PREF_SIZE);

warnings.getStyleClass().add("warning-text");
warnings.setWrapText(true);
warnings.setMaxWidth(520);
warnings.setMinHeight(Region.USE_PREF_SIZE);

VBox content = new VBox(10, grid, new HBox(reset), errors, warnings);
getDialogPane().setContent(content);
ButtonType apply = new ButtonType(applyText, ButtonBar.ButtonData.OK_DONE);
ButtonType cancel = new ButtonType("Отмена", ButtonBar.ButtonData.CANCEL_CLOSE);
getDialogPane().getButtonTypes().addAll(apply, cancel);
getDialogPane().getStylesheets().add(MazeApp.stylesheet());

applyButton = (Button) getDialogPane().lookupButton(apply);
applyButton.setId("apply");
applyButton.addEventFilter(ActionEvent.ACTION, event -> {
    if (!tryAccept()) {
        event.consume(); // есть ошибки — окно остаётся открытым
    }
});
setResultConverter(button -> button == apply ? accepted : null);
}

private void fill(Map<String, String> texts) {
    texts.forEach((key, text) -> {
        switch (inputs.get(key)) {
            case CheckBox box -> box.setSelected(Boolean.parseBoolean(text));
            case TextField field -> field.setText(text);
            case null, default -> {
            }
        }
    });
}

private boolean tryAccept() {
    Map<String, String> texts = new HashMap<>();
    inputs.forEach((key, input) -> texts.put(key, switch (input) {
        case CheckBox box -> Boolean.toString(box.isSelected());
        case TextField field -> field.getText();
        default -> "";
    }));
    inputs.values().forEach(input -> input.getStyleClass().remove("invalid"));

    switch (parser.apply(texts)) {
        case FormResult.Valid<T>(T value) -> {
            errors.setText("");
            List<String> list = warningsOf.apply(value);
            if (!list.isEmpty() && !value.equals(warned)) {
                // первое нажатие: предупредить и дать передумать; второе — применить
                warned = value;
                warnings.setText(String.join("\n", list));
                // короткая надпись: кнопки диалога одной ширины, длинная обрезалась бы
                applyButton.setText("Всё равно");
                return false;
            }
            accepted = value;
            return true;
        }
        case FormResult.Invalid<T>(List<Problem> problems) -> {
            warned = null;
            warnings.setText("");
        }
    }
}

```

```

        applyButton.setText(applyText);
        showProblems(problems);
        return false;
    }
}

private void showProblems(List<Problem> problems) {
    StringBuilder text = new StringBuilder();
    for (Problem problem : problems) {
        text.append("• ").append(problem.message()).append('\n');
        Control input = inputs.get(problem.field());
        if (input != null && !input.getStyleClass().contains("invalid")) {
            input.getStyleClass().add("invalid");
        }
    }
    errors.setText(text.toString().strip());
}
}

```

Файл src/main/resources/maze/view/app.css

```

/* Светлый плоский стиль: белые поверхности, тонкие границы, без теней и градиентов */
.root {
    /* PT Sans — шрифт, созданный под кириллицу: системный шрифт Mac в JavaFX рвёт буквы,
    жирный Arial тяжёлый.
    Если шрифта нет (не macOS), JavaFX возьмёт шрифт по умолчанию. */
    -fx-font-family: "PT Sans";
    -fx-font-size: 13px;
    -fx-background-color: #ffffff;
    -fx-base: #f7f6f3;
    -fx-accent: #185fa5;
    -fx-focus-color: #378add;
    -fx-faint-focus-color: transparent;
}

.toolbar-row {
    -fx-padding: 8 12 8 12;
    -fx-border-color: #e4e2dc;
    -fx-border-width: 0 0 1 0;
}

.title {
    -fx-font-size: 15px;
    -fx-font-weight: bold;
}

.muted {
    -fx-text-fill: #6b6a65;
}

.panel {
    -fx-padding: 12;
    -fx-border-color: #e4e2dc;
    -fx-border-width: 0 0 0 1;
}

.editor-bar {
    -fx-padding: 6 8 6 8;
    -fx-background-color: #f1efe8;
    -fx-background-radius: 8;
}

.legend {
    -fx-text-fill: #6b6a65;
}

.legend .label {
    -fx-text-fill: #6b6a65;
}

```

```

    -fx-font-size: 12px;
}

.button, .toggle-button, .text-field {
    -fx-background-radius: 6;
}

.stat {
    -fx-background-color: #f1efe8;
    -fx-background-radius: 8;
    -fx-padding: 6 10 6 10;
}

.stat-name {
    -fx-text-fill: #6b6a65;
    -fx-font-size: 11px;
}

.stat-value {
    -fx-font-size: 17px;
    -fx-font-weight: bold;
}

.status {
    -fx-text-fill: #0f6e56;
    -fx-font-weight: bold;
}

.message {
    -fx-text-fill: #854f0b;
}

.section {
    -fx-font-weight: bold;
    -fx-padding: 8 0 0 0;
}

.warning-text {
    -fx-text-fill: #854f0b;
}

.error-text {
    -fx-text-fill: #a32d2d;
}

.text-field.invalid, .checkbox.invalid {
    -fx-border-color: #e24b4a;
    -fx-border-width: 2;
    -fx-border-radius: 6;
}

/* График: бледная линия — каждая попытка, зелёная — среднее */
.chart {
    -fx-padding: 0;
}

.chart-plot-background {
    -fx-background-color: #ffffff;
}

.chart-vertical-grid-lines, .chart-horizontal-grid-lines {
    -fx-stroke: #eeede8;
}

.default-color0.chart-series-line { -fx-stroke: #c9c7bf; -fx-stroke-width: 1px; }
.default-color1.chart-series-line { -fx-stroke: #1d9e75; -fx-stroke-width: 2.5px; }

```

```
.default-color0.chart-legend-item-symbol { -fx-background-color: #c9c7bf; }  
.default-color1.chart-legend-item-symbol { -fx-background-color: #1d9e75; }
```